SAFEHER: AI-POWERED SAFE TRAVEL COMPANION FOR WOMEN

A MINI PROJECT REPORT

Submitted by

ANANTHALAKSHMY R

MLM25MCA-2019

To

the APJ Abdul Kalam Technological University
in Partial fulfillment of the requirements for the
award of the degree

of

MASTER OF COMPUTER APPLICATIONS

Under the Guidance of

Ms. ANJU JOHN

(Assistant Professor, Department of Computer Applications)

DEPARTMENT OF COMPUTER APPLICATIONS

MANGALAM COLLEGE OF ENGINEERING, ETTUMANOOR

(Affiliated to APJ Abdul Kalam Technological University)

OCTOBER 2026

MANGALAM COLLEGE OF ENGINEERING
Accredited by NAAC& ISO 9001:2000 Certified Institution
DEPARTMENT OF COMPUTER APPLICATIONS

VISION

To become a centre of excellence in computer applications,competent in the global ecosystem with technical knowledge,innovation with a sense of social commitment.

MISSION

- To serve with state of the art education,foster advanced research and cultivate innovation in the field of computer applications.
- To prepare learners with knowledge skills and critical thinking to excel in the technological landscape and contribute positively to society.

Program Educational Objectives

- PEO I :Graduates will possess a solid foundation and in-depth understanding of computer applications and will be equipped to analyze real-world problems, design and create innovative solutions, and effectively manage and maintain these solutions in their professional careers.
- PEO II: Graduates will acquire technological advancements through continued education, lifelong learning and research, thereby making meaningful contributions to the field of computing.
- PEO III: Graduates will cultivate team spirit, leadership, communication skills, ethics, and social values, enabling them to apply their understanding of the societal impacts of computer applications effectively.

Program Specific Outcomes

- **PSO I**: Apply advanced technologies through innovations to enhance the efficiency of design and development.
- **PSO II**: Apply the principles of computing to analyze, design and implement sustainable solutions for real world challenges.

MAPPING OF PO-PSO-SDG

1. MAPPING WITH PROGRAM OUTCOMES (POs):

SL.NO	POs ADDRESSED	RELEVANCE TO PROJECT
1	PO1	Mathematics and programming fundamentals were applied to build the route-risk scoring logic, geofencing distance calculations, weighted combination of safety signals and the relational database schema of the platform.
2	PO2	Existing navigation, SOS-only and accommodation booking applications were reviewed to identify their gaps for women travellers, and these gaps were formulated as requirements for one unified safety platform.
3	PO3	A complete web application combining safe route suggestion, verified stays, emergency SOS alerts and community ratings was designed and developed as an innovative solution to the safety problems women face while travelling.
4	PO4	Modern tools such as React.js, Node.js, Express.js, MySQL, the Google Maps Platform and Bluetooth/BLE wearable integration were selected and applied, and their limitations were studied and documented in the report.
5	PO5	The project was carried out individually under faculty supervision using an agile, iterative approach, with progress presented and reviewed at regular guide meetings.
6	PO6	A work breakdown structure covering requirement analysis, design, module development and testing was scheduled across the semester, and free and open-source components were used to keep the project cost low.
7	PO8	Developing route-risk prediction, live tracking and wearable integration required continuous learning of mapping APIs and current safety-technology practices beyond the prescribed syllabus.

LIST OF PROGRAM OUTCOMES (POs):

PO1 (**Foundation Knowledge**): Apply knowledge of mathematics, programming logic and coding fundamentals for solution architecture and problem solving.

PO2 (**Problem Analysis**): Identify, review, formulate and analyze problems for primarily focusing on customer requirements using critical thinking frameworks.

PO3 (**Development of Solutions**): Design, develop and investigate problems with an innovative approach for solutions incorporating ESG/SDG goals.

PO4 (**Modern Tool Usage**): Select, adapt and apply modern computational tools such as development of algorithms with an understanding of the limitations including human biases.

PO5 (**Individual and Teamwork**): Function and communicate effectively as an individual or a team leader in diverse and multidisciplinary groups. Use methodologies such as agile.

PO6 (**Project Management and Finance**): Use the principles of project management such as scheduling, work breakdown structure and be conversant with the principles of finance for profitable project management.

PO7 (**Ethics**): Commit to professional ethics in managing software projects with financial aspects. Learn to use new technologies for cyber security and insulate customers from malware.

PO8 (**Life-long learning**): Change management skills and the ability to learn, keep up with contemporary technologies and ways of working.

2. MAPPING WITH PROGRAM SPECIFIC OUTCOMES (PSOs):

SL.NO	PSOs ADDRESSED	RELEVANCE TO PROJECT
1	PSO 1 – Advanced technologies for efficient design development	AI-assisted route-risk scoring, wearable Bluetooth/BLE integration and a real-time SOS pipeline were combined to obtain a usable safety companion on standard web infrastructure.
2	PSO 2 – Computing principles for sustainable real-world solutions	The platform answers a pressing societal need, women's travel safety, through a layered React/Node/MySQL architecture that can be extended to new cities by updating crime and location datasets alone.

LIST OF PROGRAM SPECIFIC OUTCOMES (PSOs):

PSO 1: Apply advanced technologies through innovations to enhance the efficiency of design development.

PSO 2: Apply the principles of computing to analyze, design and implement sustainable solutions for real world challenges.

3. MAPPING WITH SUSTAINABLE DEVELOPMENT GOALS (SDGs):

SDG NO	SDGs ADDRESSED	RELEVANCE TO PROJECT
SDG 5	Gender Equality	Directly targets the safety and mobility of women travellers, helping remove a practical barrier to their free and confident movement in public spaces.
SDG 9	Industry, Innovation and Infrastructure	Builds a resilient, AI-assisted digital safety infrastructure and fosters innovation by unifying route intelligence, verified stays and emergency response in one open platform.
SDG 11	Sustainable Cities and Communities	Contributes to safer, more inclusive urban mobility by surfacing crime-aware routing and community-verified safe places within a city.
SDG 16	Peace, Justice and Strong Institutions	Strengthens accountable emergency response by connecting a distressed traveller instantly with trusted contacts and, where integrated, the police.

SUSTAINABLE DEVELOPMENT GOALS (SDGs):

SDG 1 – No Poverty: End poverty in all its forms everywhere.

SDG 2 – Zero Hunger: End hunger, achieve food security and improved nutrition, and promote sustainable agriculture.

SDG 3 – Good Health and Well-Being: Ensure healthy lives and promote well-being for all at all ages.

SDG 4 – Quality Education: Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all.

SDG 5 – Gender Equality: Achieve gender equality and empower all women and girls.

SDG 6 – Clean Water and Sanitation: Ensure availability and sustainable management of water and sanitation for all.

SDG 7 – Affordable and Clean Energy: Ensure access to affordable, reliable, sustainable and modern energy for all.

SDG 8 – Decent Work and Economic Growth: Promote sustained, inclusive and sustainable economic growth, full and productive employment and decent work for all.

SDG 9 – Industry, Innovation and Infrastructure: Build resilient infrastructure, promote inclusive and sustainable industrialisation and foster innovation.

SDG 10 – Reduced Inequalities: Reduce inequality within and among countries.

SDG 11 – Sustainable Cities and Communities: Make cities and human settlements inclusive, safe, resilient and sustainable.

SDG 12 – Responsible Consumption and Production: Ensure sustainable consumption and production patterns.

SDG 13 – Climate Action: Take urgent action to combat climate change and its impacts.

SDG 14 – Life Below Water: Conserve and sustainably use the oceans, seas and marine resources.

SDG 15 – Life on Land: Protect, restore and promote sustainable use of terrestrial ecosystems, manage forests sustainably, combat desertification and halt biodiversity loss.

SDG 16 – Peace, Justice and Strong Institutions: Promote peaceful and inclusive societies, provide access to justice for all and build effective, accountable and inclusive institutions.

SDG 17 – Partnerships for the Goals: Strengthen the means of implementation and revitalise the global partnership for sustainable development.

MANGALAM COLLEGE OF ENGINEERING, ETTUMANOOR
DEPARTMENT OF COMPUTER APPLICATIONS
OCTOBER 2026



DECLARATION

I hereby certify that the work which is being presented in the project entitled "SAFEHER: AI-POWERED SAFE TRAVEL COMPANION FOR WOMEN" submitted in the DEPARTMENT OF COMPUTER APPLICATIONS is an authentic record of my own work carried under the supervision of MS. ANJU JOHN, Assistant Professor, Department of Computer Applications. This study has not been submitted to any other institution or university for the award of any other degree. This report has been checked for plagiarism by the college and the similarity index is within permissible limits set by the college.

Place: *Name & Signature of Student*

Date: **ANANTHALAKSHMY R**

MANGALAM COLLEGE OF ENGINEERING, ETTUMANOOR
DEPARTMENT OF COMPUTER APPLICATIONS
OCTOBER 2026



CERTIFICATE

*Certified that the report entitled "**SAFEHER: AI-POWERED SAFE TRAVEL COMPANION FOR WOMEN**" submitted by **ANANTHALAKSHMY R MLM25MCA-2019** to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the **DEGREE OF MASTER OF COMPUTER APPLICATIONS** by APJ **ABDUL KALAM TECHONOLOGICAL UNIVERSITY** is a bonafide record of the **MINI PROJECT** work carried out by her under our guidance and supervision during the academic year 2025-27.*

PROJECT GUIDE
Ms. Anju John
Assistant Professor
Department of Computer Applications

PROJECT COORDINATOR
Ms. Athira R Kurup
Assistant Professor
Department of Computer Applications

HEAD OF THE DEPARTMENT
Ms. Divya S B
HOD & Assistant Professor
Department of Computer Applications

INTERNAL EXAMINER

ACKNOWLEDGEMENT

I am greatly indebted to the authorities of Mangalam College of Engineering for providing the necessary facilities to successfully complete my Project on the topic ***"SafeHer: AI-Powered Safe Travel Companion for Women"***.

I express my sincere thanks to **Dr. Vinodh P Vijayan**, Principal, Mangalam College of Engineering for providing the facilities to complete my Project successfully.

I thank and express my solicit gratitude to **Ms. Divya S B**, HOD & Assistant Professor, Department of Computer Applications, Mangalam College of Engineering, for her invaluable help and support which helped me a lot in successfully completing this Project work.

I express my gratitude to my Internal Guide, **Ms. Anju John**, Assistant Professor, Department of Computer Applications for the suggestions and encouragement which helped in the successful completion of my Project.

Furthermore, I would like to acknowledge with much appreciation the crucial role of the faculties especially Project coordinator, **Ms. Athira R Kurup**, Assistant Professor, Department of Computer Applications, Mangalam College of Engineering, who gave the permission to use all the required equipment and the necessary resources to complete the presentation and report.

Finally, I would like to express my heartfelt thanks to my parents who were very supportive both financially and mentally and for their encouragement to achieve my goal.

ANANTHALAKSHMY R

MLM25MCA-2019

ABSTRACT

Solo and night travel among women is rising sharply across Indian cities, yet the personal-safety tools available to a traveller have not kept pace with the risks she actually faces. Mainstream navigation applications optimise a route for distance and time, standalone panic-button applications send an alert but carry no sense of the route ahead, and hotel or PG booking platforms rarely disclose whether a stay has CCTV coverage, on-site security staff, or a women-only policy. A woman travelling alone is consequently forced to juggle several disconnected applications, none of which was designed with her specific risk profile in mind.

SafeHer is proposed as a single, unified, AI-powered web platform that brings route-risk prediction, verified accommodation discovery, one-tap emergency response and a trusted women-only community together in one place. The platform predicts the relative risk of a route by combining historical crime data, the time of day, street lighting and crowd density, and presents the traveller with a safer alternative alongside the fastest one. A directory of hotels and paying-guest accommodations is filtered by safety amenities such as CCTV coverage, security personnel and women-only status, each backed by community ratings on lighting, crowd presence and accessibility. In an emergency, a one-tap or voice-activated SOS immediately shares the user's live location with her registered emergency contacts and, where integrated, the nearest police control room, and a paired wearable device allows the same alert to be raised discreetly and hands-free when reaching for a phone is not possible.

The system is built on a modern, scalable stack: React.js on the client, Node.js with Express.js on the server, and MySQL as the relational data store, with the Google Maps Platform providing routing and places data. An administrator console supports verification of accommodation listings, moderation of community posts and reviews, monitoring of live SOS alerts, and management of the underlying safety datasets, so that the quality of the information the traveller relies on is actively maintained rather than left to crowdsourcing alone. Twelve interlinked entities in the data model, spanning users, emergency contacts, trips, live-tracking sessions, SOS alerts and community content, allow the six core modules of the platform, namely User and Authentication, Trip and Route, Safe Places, Emergency, Notifications, and Admin and AI, to operate as one coherent experience rather than as a collection of separate features.

This mini project report documents the requirement analysis, the feasibility study, the system design captured through data flow, use case, class, state, sequence and entity-relationship diagrams, and the implementation approach adopted for **SafeHer**. The resulting design demonstrates that gender-specific travel safety can be delivered through a single, proactive and

always-on companion built entirely from open, well-supported web technologies, in place of the fragmented and reactive tools a woman traveller is compelled to rely on today.

Keywords: Women's safety, route-risk prediction, SOS emergency response, live location sharing, verified accommodation, community safety rating, wearable integration, React.js, Node.js, MySQL.

Mapping with Sustainable Development Goals (Mention the Goal)	SDG 5 Gender Equality SDG 9 Industry, Innovation and Infrastructure SDG 11 Sustainable Cities and Communities SDG 16 Peace, Justice and Strong Institutions

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ACKNOWLEDGEMENT	**I**
	ABSTRACT	**II**
	LIST OF TABLES	**IV**
	LIST OF FIGURES	**V**
	LIST OF ABBREVIATIONS	**VI**
Chapter 1	**INTRODUCTION**	**1**
1.1	Background	1
1.2	Motivation	2
1.3	Scope and Objectives	3
1.4	Problem Statement	5
Chapter 2	**REQUIREMENTS AND ANALYSIS**	**6**
2.1	Existing System and Problem Statement	6
2.2	Solution Overview	6
2.3	Feasibility Study	7
2.4	Requirement Specification	10
2.5	Risk Analysis and Mitigation	12
Chapter 3	**LITERATURE REVIEW**	**14**
3.1	Women Travel Safety Companion: A Real-Time Monitoring and Alert System (2026)	14
3.2	SafeRoutes: Charting a Secure Path (2024)	14
3.3	Safety for HER: A Systematic Approach with Coalescence of Technology and Citizens (2021)	14
3.4	Safarion: An AI-Driven Safety and Travel Companion for Solo Explorers (2026)	15
3.5	Women Safety Mobile Application Using ESP32 Camera and Real-Time Crime Data Analysis (2026)	15
3.6	The Women Safety Tracker App (2026)	15
3.7	Android-Based Women Safety Application with Real-Time Alerts and Location Tracking (2025)	15
3.8	Predictive Hotspot Mapping for Data-driven Crime Prediction (2026)	16
3.9	Crime Hotspot Prediction Using Deep Graph Convolutional Networks (2025)	16
3.10	Google Maps Platform, Routes API Documentation	16
3.11	Summary of the Literature	16

3.12	Research Gap Identified	17
3.13	Lessons Carried into the Design	18
Chapter 4	**DIAGRAMS**	**19**
4.1	Data Flow Diagrams	19
4.1.1	DFD Level 0 — Context Diagram	19
4.1.2	DFD Level 1 — Admin	19
4.1.3	DFD Level 1 — User	20
4.1.4	Data Stores	21
4.2	Use Case Diagram	22
4.3	Class Diagram	23
4.4	State Diagram	24
4.5	Activity Diagram	24
4.6	Sequence Diagram	25
Chapter 5	**METHODOLOGY**	**27**
5.1	Module Descriptions	27
5.2	Algorithms	32
5.3	Architecture	34
5.4	Case Study	35
5.5	Technology Stack	36
5.6	Design Decisions and Rationale	36
5.7	Security Architecture	37
Chapter 6	**IMPLEMENTATION DETAILS**	**39**
6.1	Implementation Procedures	39
6.2	Testing Methods and Results	40
6.3	Organisation of the Database Schema	42
6.4	Deployment Considerations	43
Chapter 7	**ADVANTAGES AND DISADVANTAGES**	**44**
7.1	Advantages	44
7.2	Disadvantages	44
7.3	Overall Assessment	45
7.4	Mitigation of the Disadvantages	45
Chapter 8	**RESULT AND DISCUSSION**	**47**
8.1	Analysis	47
8.2	Comparison with Existing Solutions	48
8.3	Key Observations	48
8.4	Objective-wise Discussion	49
8.5	Privacy and Ethical Evaluation	50
Chapter 9	**CONCLUSION**	**51**

9.1	Summary	51
9.2	Limitations	51
9.3	Future Scope	51
9.4	Applications	52
9.5	Concluding Remarks	52
9.6	Recommendations for Further Work	53
Chapter 10	**APPENDICES**	**54**
10.1	Sample Code	54
Chapter 11	**REFERENCES**	**58**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Comparison of existing women-safety travel options	10
2.2	Minimum and recommended hardware requirements	10
2.3	Software requirements and versions	10
2.4	Functional requirement summary	11
3.1	Consolidated summary of the reviewed literature	16
4.2	Data stores used in the Level 1 diagrams	22
5.1	Summary of implemented modules and responsibilities	27
5.2	Route-risk signal weights used in the scoring module	32
5.3	Technologies used and their roles	36
6.1	Application database tables and key columns	40
6.2	Unit and integration test suite results	41
6.3	Representative system test cases	41
6.4	Entity groups in the database schema	42
8.1	Achievement against the objectives stated in Chapter 1	47

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1.1	Data flow diagram — Level 0 (context diagram)	19
4.1.2	Data flow diagram — Level 1 (Admin)	20
4.1.3	Data flow diagram — Level 1 (User)	21
4.2	Use case diagram of SafeHer	23
4.3	Class diagram of domain entities	24
4.4	State diagram of the SafeHer lifecycle	24
4.5	Sequence diagram of a single SOS request	25
4.6	Entity relationship diagram of the database schema	26
5.1.1	User login	28
5.1.2	User Registration	28
5.1.3	User Dashboard	28
5.1.4	Trip & Journeys	29
5.1.5	Route Predictions	29
5.1.6	Safe Place	30
5.1.7	SOS alert	30
5.1.8	Notifications	31
5.1.9	Admin login interface	31
5.1.10	Admin dashboard	32
5.3.1	System architecture of SafeHer	35

LIST OF ABBREVIATIONS

ABBREVIATION		FULL FORM
AI	-	Artificial Intelligence
API	-	Application Programming Interface
BLE	-	Bluetooth Low Energy
CCTV	-	Closed-Circuit Television
CRUD	-	Create, Read, Update, Delete
DFD	-	Data Flow Diagram
ER	-	Entity Relationship
GPS	-	Global Positioning System
HTTP	-	Hypertext Transfer Protocol
JSON	-	JavaScript Object Notation
JWT	-	JSON Web Token
LBS	-	Location-Based Services
MCA	-	Master of Computer Applications
ML	-	Machine Learning
NLP	-	Natural Language Processing
ORM	-	Object Relational Mapper
PG	-	Paying Guest accommodation
REST	-	Representational State Transfer
SDG	-	Sustainable Development Goal
SOS	-	Save Our Souls (distress signal)
SQL	-	Structured Query Language
SSL/TLS	-	Secure Sockets Layer / Transport Layer Security
UI	-	User Interface
UML	-	Unified Modeling Language
UX	-	User Experience

CHAPTER 1
INTRODUCTION

1.1 Background

Solo and night travel among women has grown steadily across Indian cities over the last decade, driven by longer working hours, late-shift jobs, higher education away from home and greater independence in travel choices. This growth has not been matched by a corresponding growth in the safety tools available to the traveller herself. Mainstream navigation applications such as Google Maps are built to optimise a route for distance and time; they have no notion of street lighting, crowd density or the crime history of a locality, and therefore cannot distinguish a well-lit main road from a poorly lit shortcut that happens to be a few minutes faster.

Personal-safety applications exist as a separate category, typically offering a panic button that sends an alert with the user's location to a short list of contacts. These applications are reactive rather than predictive: they help after something has already gone wrong, but they say nothing about the route a woman is about to walk or the safety profile of the hotel or PG she is about to check into. Hotel and PG booking platforms, in turn, rarely disclose gender-specific safety information such as CCTV coverage, the presence of security staff, or a women-only policy, leaving the traveller to gamble on a listing's photographs and star rating alone.

The result is a fragmented experience: a woman planning a solo trip today must combine a mapping application, a separate SOS application, a booking website and, often, word-of-mouth advice from other women, none of which talk to each other. SafeHer is conceived to close this gap by bringing route-risk prediction, verified accommodation discovery, one-tap emergency response and a trusted community into a single, coherent, AI-assisted platform.

The problem is as much one of information as of technology. A traveller deciding between two routes late in the evening typically has only an estimated travel time to go on, while the facts that would actually influence her choice, how well lit the road is, whether shops are still open, whether the stretch has a history of reported incidents, are scattered across local knowledge, news reports and the experience of other women. Making that knowledge available at the moment of decision, and in a form that can be compared at a glance, is the first thing a safety-oriented travel platform has to do.

Accommodation raises a parallel difficulty. A woman arriving in an unfamiliar city, often after dark, must choose where to stay on the strength of photographs and generic star ratings that say little about the aspects of a stay that matter most to her, such as whether the entrance is

monitored, whether security staff are present at night and whether the property admits only women. Hostels and paying-guest accommodations, which are the most common choice for students and working women, are the least consistently documented of all. SafeHer treats the stay as part of the journey rather than as a separate decision, and brings it under the same verification and rating discipline as the route.

Technology has changed the way people move through cities, but its benefits have not been shared equally. Smartphones, mobile data and online maps have made it easy to plan a trip, book a stay and share a location, yet most of these conveniences were designed around the average traveller and not around the specific concerns of a woman travelling alone. For her, safety is a question that begins before the journey does: which road to take, which part of town to avoid after dark, and where it is wise to stay. Bringing these questions into the planning stage, and answering them with data rather than guesswork, is the starting point for the work reported here.

1.2 Motivation

The motivation for this project came from observing how differently women and men experience the same city after dark. A route that feels entirely ordinary to one traveller can feel genuinely unsafe to another, and yet the tools both use to navigate that route are identical. This observation motivated the central design decision of SafeHer: safety-relevant context, lighting, crowd density, time of day and locality crime history, should be a first-class input to route planning, not an afterthought bolted onto a generic map.

A second motivation was the recognition that an SOS button alone is not enough. An emergency alert is valuable, but by the time it is needed the traveller may already be in a difficult situation. A system that also helps a woman choose a safer route in advance, and choose accommodation that has been verified and rated by other women, addresses the problem earlier in the journey rather than only at its worst moment.

A third motivation was practical: most existing safety applications are narrow point solutions built by different vendors with no shared data or design philosophy. Building SafeHer as a single platform, with one consistent data model spanning users, trips, places and alerts, makes it possible for the route module, the accommodation module and the emergency module to reinforce one another, for example by letting a live trip automatically arm the SOS flow with the user's current location rather than requiring it to be re-entered during a crisis.

The project was also motivated by a design question rather than a purely technical one: how much of a safety tool's value comes from its individual features, and how much from the fact

that they share context? A route plan that knows the traveller's destination, an SOS that knows her current trip and a contact list that is already notified through the same channel are each modest on their own. Combined, they remove the steps a person under stress would otherwise have to perform by hand, which is where a safety tool is most likely to fail in practice.

There was also a social dimension to the motivation. Many women already follow informal safety habits, such as sharing a live location with a friend, avoiding certain lanes after dark or choosing accommodation recommended by someone they know. These habits are sensible, but they depend on individual effort and word of mouth, which neither scale nor help a traveller who is new to a city. The project was motivated by the belief that this informal knowledge could be organised, verified and delivered to every user through software, so that a safe journey depends less on luck and personal contacts and more on reliable, shared information.

Finally, the project offered an opportunity to apply the course material of the Master of Computer Applications programme to a problem with a clear social purpose. Database design, web application development, API integration, algorithm design, software testing and project management all had a part to play, and SafeHer therefore served as an integrating exercise across the semester rather than an isolated programming task.

1.3 Scope and Objectives

1.3.1 Scope

- A cross-platform web application built with React.js, Node.js/Express and MySQL, aimed at women travellers using solo trips, daily commutes, night travel and hostel/PG stays in urban areas.
- Covers route safety scoring, verified accommodation discovery, SOS emergency response and a women-only community, together with an admin/AI backend for verification, moderation and model updates.
- Route-risk prediction uses historical crime data, time of day, lighting and crowd density as inputs, combined with Google Maps Platform routing data.
- Emergency response supports one-tap and voice-activated SOS with live location sharing to registered contacts and, where integrated, the police, together with hands-free wearable-device triggering.
- Explicitly outside scope for this mini project are a native mobile application, offline operation without network connectivity, and integration with a live official police dispatch system, which are noted as future scope in Chapter 9.

1.3.2 Objectives

- To design a route-risk scoring model that combines historical crime data, time of day, street lighting and crowd density into a single, explainable safety score for a candidate route.
- To build a verified directory of women-friendly hotels and PGs, filterable by CCTV coverage, security staff and women-only status, backed by community ratings.
- To implement a one-tap and voice-activated SOS mechanism that shares live location with emergency contacts and police, and to support the same trigger from a paired wearable device.
- To build a women-only community layer where users can rate and comment on the lighting, crowd presence and accessibility of places they have visited.
- To provide a real-time crime-zone heatmap and an AI chatbot that offers safety guidance and answers routine safety questions.
- To build an admin and AI module for verifying accommodation listings, moderating community content, monitoring SOS alerts and reviewing platform analytics.
- To deliver the whole capability through a responsive, authenticated web interface with a live map tracker, notification centre and audit logging.

1.3.3 Assumptions and Constraints

The design and evaluation of SafeHer rest on a small number of explicit assumptions and are bounded by constraints that are worth stating at the outset, since they determine how the results in Chapter 8 should be read.

- The user has a smartphone or laptop with a modern browser, a working internet connection and, for live tracking, has allowed the browser to read the device location.
- Emergency contacts have agreed in advance to be listed by the user and are able to receive a message and open a link on their own devices.
- Historical crime, lighting and crowd data are available for the area of interest; in this mini project they take the form of an illustrative dataset rather than an official municipal record.
- The project was carried out by a single developer within one semester, which limited the breadth of integration, particularly with external agencies and certified hardware.
- Free and open-source components were preferred throughout so that the platform could be built and demonstrated without licensing cost.

1.4 Problem Statement

Women travelling alone or at night face heightened safety risks that the existing generation of travel and safety applications addresses only partially and only in isolation. Navigation applications optimise for speed rather than safety, standalone SOS applications react only after an incident has begun, and accommodation platforms disclose nothing about the gender-specific safety of a stay. The traveller is left to assemble her own safety picture from several disconnected sources, under time pressure and often at the exact moment she can least afford the distraction.

What is required, therefore, is a single platform that predicts route risk before a journey begins, verifies and surfaces accommodation that is genuinely safer for a woman, keeps a trusted circle of contacts and, where relevant, the police instantly reachable during an emergency, and does all of this from one authenticated account rather than several disconnected applications. SafeHer is designed to be that platform.

Stated in terms of requirements, the problem has four parts. The first is prediction: the traveller must be told something about the safety of a route before she commits to it. The second is verification: claims about the safety of a place to stay must be checked by someone other than the place itself. The third is response: if something goes wrong, the means of asking for help must be available through whichever input the situation allows, and the people who receive the alert must be able to follow the user's position rather than only read a single message. The fourth is trust: all of this must be delivered by a system that protects the sensitive location data it handles and that can account for the actions of its administrators.

The consequences of this gap are practical as well as emotional. A traveller who cannot judge the safety of a route or a stay may choose a longer and costlier alternative, change her plans, or avoid travelling at certain hours altogether, which limits her access to work, education and independence. An effective solution therefore has to be usable under pressure, ask very few steps of the user, and be transparent enough that she can trust the score or verification badge it shows. These expectations, together with the four requirements above, define what SafeHer must deliver and form the basis for the objectives listed in Section 1.3.

The scope of the problem is deliberately bounded. SafeHer does not attempt to prevent crime, replace the police or guarantee the safety of any journey. Its purpose is to improve the information available to the traveller before and during a journey and to shorten the time between a woman needing help and the right people knowing about it.

CHAPTER 2
REQUIREMENTS AND ANALYSIS

2.1 Existing System and Problem Statement

In the absence of a unified platform, a woman planning a trip today relies on a combination of general-purpose tools, each with a specific and well-documented limitation.

- Navigation apps (Google Maps and similar): optimise for distance and time, not for lighting, crowd density or crime history.
- Standalone SOS apps (bSafe, Himmat, Circle of 6 and similar): send location alerts on demand, but offer no route intelligence or stay verification.
- Hotel booking platforms (OYO, Booking.com and similar): list stays without gender-specific safety verification such as CCTV coverage or women-only status.
- Fragmented experience: users must switch between maps, safety apps and booking sites, with no single trusted platform tying the three together.
- No predictive intelligence: none of the existing tools combine AI-driven route-risk prediction with community-sourced safety data.

These limitations are not failures of the individual tools, each of which is designed for a different purpose. A navigation application is judged by how quickly it gets a user to a destination, a booking platform by how many stays it can fill and an SOS application by how quickly it can send a message. None is judged by the broader question of whether a woman travelling alone was safe and felt safe, and so none is built to answer it.

The practical consequence is that the burden of integration falls on the traveller. She must compare routes in one application, search for a stay in another, share her live location through a third and remember to inform her contacts before she leaves. Every additional step is an opportunity for the process to be abandoned, particularly late in the day or when the traveller is tired, and the information passed between the applications is limited to what the user can copy across by hand.

2.2 Solution Overview

SafeHer is proposed as one unified, AI-driven platform covering routes, stays, community and emergency response. Route-risk scoring is produced by an AI/ML model that combines historical crime data, time of day, lighting and crowd density; a verified directory of women-friendly hotels and PGs surfaces CCTV coverage, security staff and women-only status; one-tap and voice-activated SOS shares live location with contacts and police; a women-only

community layer contributes crowdsourced ratings on lighting, crowds and accessibility; a paired wearable device gives a discreet, hands-free way to trigger SOS; and an admin and AI module keeps verification, moderation and model updates under active, accountable control rather than leaving them to unmoderated crowdsourcing.

The platform additionally exposes real-time crime-zone heatmaps and an AI-powered chatbot for instant safety guidance, so that a user who has a routine safety question, for example about a locality she is unfamiliar with, does not need to leave the application to find an answer.

The solution is organised around three ideas. The first is that safety information should be attached to the decision it informs: the route score is shown next to the route, and the verification badge next to the stay. The second is that emergency response should reuse the context already held by the platform, so that an SOS raised during a trip already knows who the user is, where she is and whom to tell. The third is that the quality of the information should be maintained by people as well as by algorithms, through administrator verification of listings and moderated community ratings, so that the platform does not rely on a single automated source.

From the traveller's point of view the platform is used in three stages. Before a journey she plans a trip, compares the scored routes and, if needed, chooses a verified stay. During the journey the live map shows her position, the notification centre delivers weather, traffic and night-travel warnings, and the SOS control remains available. After the journey the trip is added to her private history and she can rate the places she visited so that the next traveller benefits from her experience.

2.3 Feasibility Study

2.3.1 Technical Feasibility

The platform is built on a proven, well-supported stack: React.js for the client, Node.js with Express.js for the REST API server, and MySQL as the relational data store. The Google Maps Platform supplies routing, places and geocoding data, while the machine-learning models used for risk scoring are exposed to the rest of the system through REST APIs, keeping the intelligence layer decoupled from the web layer. Wearable-device integration uses Bluetooth Low Energy and standard SDKs, and cloud hosting is chosen so that scalable, real-time SOS and location updates remain responsive as the user base grows.

The main technical risks were examined early and checked against the chosen stack. Browser geolocation supplies the position needed for live tracking without any installation, the REST design keeps the client and the server independent so that either can be changed without

disturbing the other, and the relational schema provides the referential integrity needed to tie users, contacts, trips, alerts and reviews together. The one capability a browser cannot guarantee is continuous background location access on a locked phone; this is recorded openly as a limitation and carried into the future scope.

2.3.2 Economic Feasibility

Open-source frameworks, React, Node.js and MySQL, keep licensing cost minimal, and pay-as-you-go pricing for the Maps API and cloud hosting scales naturally with the number of active users rather than requiring a large upfront investment. The low initial investment can be recovered through a freemium model, verified-partner listing fees from accommodation providers, or B2B safety-data licensing, and the combined platform is markedly cheaper to build and operate than three separate applications for maps, safety and booking would be.

The recurring costs are limited to map and place requests, hosting and the channel used to notify contacts, and none of them arises until the platform is actually used. The free or low-cost tiers that such services normally offer are sufficient for development and demonstration, and the cost per active user can be measured and managed once real usage begins.

2.3.3 Operational Feasibility

The interface follows conventions already familiar from everyday mobile applications, register, log in, plan a journey, tap SOS if needed, which keeps the learning curve low for a first-time user. An admin module keeps verification and moderation manageable for a small operations team, a 24/7 automated SOS pipeline reduces dependency on manual intervention at the moment it matters most, and the platform is designed to scale to new cities simply by updating the underlying crime and location datasets rather than by re-engineering the application.

Operational readiness also depends on people. An administrator needs a short and clear procedure for reviewing a listing, resolving a report and closing an SOS alert, and the admin console presents each of these as a queue with a single action so that the procedure can be learned quickly. The emergency contact is the other half of the system, so the message sent to her is kept brief and carries only the tracking link and the name of the person who needs help.

2.3.4 Legal and Ethical Feasibility

Legal feasibility. Because the platform processes personal data, including names, contact details and live location, it has to be operated in line with the data-protection law of the country in which it is used. In India this means the Digital Personal Data Protection Act, 2023, which expects lawful and clearly consented processing, collection limited to a stated purpose, and a

way for the person concerned to withdraw consent or have her data erased. SafeHer follows this pattern: the user enters her own details and chooses her own contacts, location is used only for a trip or an alert that she has started, and her records belong to her account. The third-party services are used within the providers' terms, the open-source libraries are used under licences that permit project work of this kind, and the platform presents itself as an aid and not as a substitute for the police or emergency services, so that no user is led to believe that an alert guarantees a response.

Ethical feasibility. Live-location sharing and emergency-contact data are sensitive by nature, so access to a user's location is limited to her own registered contacts and, during an active SOS, to the operator console; a user's trip history is private to her account. Accommodation listings are verified by an administrator before they are shown as "verified" to a searching user, which limits the platform's exposure to false safety claims, and community reviews are moderated to prevent misuse of the rating system. The route-risk score also raises a question of fairness, since a score built on reported crime could unfairly label a locality; it is therefore shown as guidance together with the signals behind it and not as a verdict. Emergency contacts are the other people affected by the platform, and the design respects them by letting the user decide who is listed and by keeping their details visible only to her.

2.3.5 Schedule Feasibility

The work was decomposed into six increments: requirement analysis and data-model design, the authentication and profile module, the trip and route module with map integration, the safe-places and emergency modules, the community and notification modules, and finally the admin console together with testing and documentation.

Each increment ended with a review before the next began, so that a problem in an earlier module, for example in the data model, was found before later modules were built on top of it. The authentication module came first because every other module depends on a signed-in user, and the admin console and documentation came last because they depend on all the others. A small reserve of time was kept for testing and for correcting defects found during the guide's reviews.

Table 2.1: Comparison of existing women-safety travel options

Criterion	Navigation apps	Standalone SOS apps	Hotel booking platforms	SafeHer (proposed)
Route-risk awareness	None (distance/time only)	Not applicable	Not applicable	Crime, lighting, time, crowd-aware
Verified women-friendly stays	No	No	Rarely / self-declared	Admin-verified, filterable
Emergency SOS with live sharing	No	Yes, basic	No	One-tap / voice + wearable
Community safety ratings	Limited (traffic only)	No	Generic reviews	Lighting, crowd, accessibility ratings
Single integrated platform	No	No	No	Yes

2.4 Requirement Specification

2.4.1 Hardware Requirements

Table 2.2: Minimum and recommended hardware requirements

Component	Minimum	Recommended
Processor	Dual-core x86-64, 2.0 GHz	Quad-core x86-64, 2.5 GHz or higher
System memory	4 GB RAM	8 GB RAM or more
Free disk space	2 GB (application and dependencies)	10 GB including media store and logs
Network	Broadband, required for maps and live tracking	Broadband with low latency
Client device	Any modern smartphone or laptop with a browser	GPS-enabled smartphone for live tracking

2.4.2 Software Requirements

Table 2.3: Software requirements and versions

Category	Component	Version
Language runtime	Node.js	18.x LTS or later
Frontend framework	React.js	18.x
Backend framework	Express.js	4.x
Database	MySQL	8.0 or later
ORM	Sequelize / Prisma	Current stable
Maps and routing	OpenStreetMap + OSRM	Current
Authentication	JSON Web Tokens (JWT), bcrypt	Current stable

Category	Component	Version
Real-time communication	Socket.IO	4.x
Push notifications	Fast2sms API	Current
Version control	Git	Any current release

2.4.3 Functional Requirements

Table 2.4: Functional requirement summary

ID	Requirement	Priority
FR-01	The system shall allow a visitor to register a safety profile and an existing user to log in and log out securely.	High
FR-02	The system shall allow a user to add, edit and delete emergency contacts.	High
FR-03	The system shall compute a safety-scored route between a source and a destination using crime, lighting, time and crowd signals.	High
FR-04	The system shall allow a user to trigger a one-tap or voice-activated SOS alert that shares live location with emergency contacts and police.	High
FR-05	The system shall accept an SOS trigger from a paired wearable device.	Medium
FR-06	The system shall let a user search and filter verified hotels/PGs by CCTV, women-only status and security staff.	High
FR-07	The system shall allow a user to submit a review and safety rating for a place she has visited.	Medium
FR-08	The system shall display a real-time crime-zone heatmap and nearby police/hospital locations on a map.	Medium
FR-09	The system shall provide an AI chatbot that answers routine safety questions and offers safety tips.	Medium
FR-10	The system shall give an administrator tools to verify places, moderate reviews, monitor SOS alerts and view analytics.	High
FR-11	The system shall maintain a private trip and SOS history for each user.	Medium
FR-12	The system shall log every administrative action to an audit trail.	Low

2.4.4 Non-Functional Requirements

Alongside the functional requirements of Table 2.4, the platform is expected to meet a number of non-functional requirements that describe how well, rather than whether, it performs its functions.

- **Performance:** the route-planning request should return scored routes within a few seconds on a normal broadband connection, and an SOS trigger should reach the server and begin notifying contacts without a perceptible delay.

- **Reliability:** a failed notification should be retried and its status recorded, so that a transient network fault does not silently discard an alert.
- **Security:** passwords are stored only as hashes, sessions are token-based, and administrator functions are inaccessible to ordinary user accounts.
- **Privacy:** a user's location and trip history are visible only to her, to the contacts she has registered and, during an active SOS, to the operator console.
- **Usability:** the main journeys, registering, planning a trip and raising an SOS, should be completed in a small number of steps using conventions familiar from everyday applications.
- **Maintainability:** the route, controller and service layering should allow one module to be changed without rewriting the others, and the scoring weights should be adjustable without changing the surrounding code.
- **Portability:** the client should run in any current browser on a phone or laptop without installation.
- **Auditability:** administrative actions should leave a record that identifies who acted, on what and when.

These requirements guided several of the design choices described in Chapter 5, particularly the layered architecture and the decision to keep the scoring weights in one clearly identified place.

2.5 Risk Analysis and Mitigation

A project of this kind carries technical, data and operational risks. They were considered during planning so that each could be addressed in the design rather than discovered late.

- **Sparse or outdated crime data:** the route-risk score depends on the data behind it. The weights are therefore kept separate from the data, community reports are used as an additional signal, and the model is described throughout as illustrative until it is trained on a larger dataset.
- **Dependency on external services:** routing and places data come from the Google Maps Platform. SafeHer requests candidate routes and re-ranks them locally, so only the retrieval step depends on the external service and the scoring logic remains under the project's control.
- **Unreliable connectivity:** SOS delivery over a weak connection may fail. Dispatch retries with an increasing delay, and the delivery status of every recipient is stored with the alert.

- **Misuse of the SOS feature:** a false alert wastes the attention of contacts. Every alert is logged against the user's account and the resolution of each alert is recorded, so that patterns can be reviewed.
- **Unverified or misleading listings:** an accommodation could claim safety features it does not have. A listing is shown as verified only after an administrator has marked it so, and community ratings provide a second check.
- **Exposure of location data:** live location is sensitive. Access is limited to registered contacts and the operator console, and a tracking link is tied to an active session that ends when the user marks herself safe.
- **Single-developer schedule:** one person completing six increments in a semester risks delay. The work was divided into incremental stages with regular guide reviews so that slippage was visible early.

The risks above are not independent of one another. Poor data quality, for instance, lowers the value of the score, and an overstated score would in turn weaken the trust a user places in the platform. The mitigations were therefore chosen to work together: the score is shown with its signals, listings carry a verification state, and every alert and administrative action is recorded, so that a weakness in one area is visible and does not silently undermine the others.

CHAPTER 3
LITERATURE REVIEW

This chapter reviews ten publications that informed the design of SafeHer. Each review notes the principal contribution of the work and the specific way it influenced a decision taken in this project.

3.1 Women Travel Safety Companion: A Real-Time Monitoring and Alert System (2026)

B. Dhanalakshmy, U. Pavitra, S. Hema and Mrs. Angayarkanni, published in the International Journal of Research Publication and Reviews, used GPS tracking and geofencing to provide live location monitoring, SOS alerts and trusted-contact notification. The authors measured alert response time and tracking accuracy and reported improved emergency response through continuous travel monitoring.

This work confirmed that GPS tracking combined with geofencing is a reliable and low-cost foundation for the live-tracking session and SOS-alert entities used in SafeHer, and its emphasis on response-time measurement shaped the decision to record alert timestamps against every SOS record.

3.2 SafeRoutes: Charting a Secure Path (2024)

Kushal Agrawal, Aviral Srivastava, Kanishk Sharma and colleagues, published in IEEE Access, applied a Gaussian Mixture Model and K-Means clustering over crime data and GPS integration to identify safe travel routes, evaluated through a safety score, clustering accuracy and route quality.

This paper is the strongest single influence on the route-risk scoring approach adopted in SafeHer: clustering historical incident locations before scoring a candidate route is the technique the Trip and Route module is built around, and its use of a cloud data pipeline motivated keeping the risk model behind a REST API rather than embedding it in the client.

3.3 Safety for HER: A Systematic Approach with Coalescence of Technology and Citizens (2021)

Logesh Rajendran and Shyam Shankaran R, presented at the IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT 2021), combined GPS-based location tracking, Location-Based Services and an SOS alert mechanism with an ICT command-and-control centre, notifying emergency contacts, nearby volunteers and police, with camera support for evidence.

This paper's inclusion of nearby volunteers alongside contacts and police informed the SafeHer data model's SOSRecipient entity, which is designed to accept more than one class of recipient rather than a single fixed contact list.

3.4 Safarion: An AI-Driven Safety and Travel Companion for Solo Explorers (2026)

Sruthy Ravi, Meya Maria Tony, Sreeya Menon P. T., Nikhil Raju and Asst. Prof. Jisha Mariam Jacob combined AI voice-activated SOS, live GPS tracking, geofencing alerts, offline emergency support and community safety reporting, and reported improvements in emergency response time, tracking accuracy and voice-SOS detection accuracy.

The demonstrated feasibility of voice-activated SOS directly motivated including a voice trigger, alongside the one-tap trigger, as a first-class input to the SafeHer Emergency module, and its community safety reporting shaped the CommunityPost and PlaceReview entities in the SafeHer schema.

3.5 Women Safety Mobile Application Using ESP32 Camera and Real-Time Crime Data Analysis (2026)

Thawale et al. combined a low-cost ESP32 camera module for visual evidence capture with real-time crime-data analysis to strengthen the evidentiary value of an SOS alert.

This paper motivated treating an SOS alert as a record that can later carry supporting evidence, not merely a location ping, and informed keeping the SOSAlert schema extensible for future attachment of captured media.

3.6 The Women Safety Tracker App (2026)

Dash et al. described a tracker application built around continuous background location logging and a simplified trusted-contact notification flow, aimed at ease of first-time use.

The paper's emphasis on a minimal, low-friction onboarding flow shaped the three-step registration used in SafeHer, full name, email/phone and password, deferring emergency-contact entry to the dashboard rather than the registration form itself.

3.7 Android-Based Women Safety Application with Real-Time Alerts and Location Tracking (2025)

Bengare et al. built a native Android application combining shake-to-trigger SOS detection with real-time location tracking, evaluated for battery efficiency and alert reliability.

While SafeHer is delivered as a web application rather than a native app, this paper's attention to alert reliability under intermittent connectivity motivated designing the SOS flow to retry delivery to recipients rather than fire-and-forget.

3.8 Predictive Hotspot Mapping for Data-driven Crime Prediction (2026)

Sriram et al. applied spatial hotspot-mapping techniques to historical crime records to predict locations of elevated risk at different times of day.

This work is the direct basis for the crime-zone heatmap feature in SafeHer and for treating time-of-day as a first-class dimension of the route-risk score rather than a static overlay.

3.9 Crime Hotspot Prediction Using Deep Graph Convolutional Networks (2025)

Zubair et al. modelled a city's street network as a graph and applied graph convolutional networks to predict crime hotspots with higher spatial precision than grid-based methods.

Although SafeHer's mini-project scope uses a lighter clustering approach for feasibility on limited data, this paper is cited in Chapter 9 as the natural upgrade path for the route-risk model once a larger, real crime dataset becomes available.

3.10 Google Maps Platform, Routes API Documentation

The official Routes API documentation describes route computation, waypoint optimisation and place-detail retrieval as REST services consumable from a Node.js backend.

This documentation was used directly to design the Trip and Route module's integration boundary: SafeHer requests candidate routes from the Routes API and re-ranks them locally using the safety score, rather than attempting to reimplement routing itself.

3.11 Summary of the Literature

Table 3.1: Consolidated summary of the reviewed literature

No.	Work	Principal Contribution	Influence on SafeHer
1	Women Travel Safety Companion (2026)	GPS + geofencing live monitoring and alerts	Live-tracking session and SOS-alert design
2	SafeRoutes (2024)	Clustering-based safe route identification	Route-risk scoring approach
3	Safety for HER (2021)	SOS to contacts, volunteers and police	Multi-recipient SOSRecipient entity
4	Safarion (2026)	Voice-activated SOS and community reporting	Voice-SOS trigger; community entities
5	ESP32 camera-based app (2026)	Visual evidence capture for SOS	Extensible SOS-alert schema

No.	Work	Principal Contribution	Influence on SafeHer
6	Women Safety Tracker App (2026)	Low-friction onboarding	Three-step registration flow
7	Android SOS + tracking (2025)	Alert reliability under poor connectivity	SOS retry-delivery design
8	Predictive Hotspot Mapping (2026)	Time-aware crime hotspot prediction	Crime-zone heatmap feature
9	Deep Graph Convolutional Hotspots (2025)	Graph-based hotspot precision	Noted future upgrade path
10	Google Maps Routes API docs	Standard REST routing and places service	Trip and Route integration boundary

3.12 Research Gap Identified

Reading the reviewed works side by side shows that each of them addresses one part of the women's travel-safety problem, while none of them, as summarised in Table 3.1, brings those parts together on a single platform. The gaps that follow from this are listed below.

- **Route intelligence and emergency response are studied separately.** SafeRoutes, predictive hotspot mapping and graph-based hotspot prediction concentrate on identifying risky areas or safer routes. Women Travel Safety Companion, Safety for HER, Safarion and the Android and ESP32-based applications concentrate on alerting and tracking once a user is already in difficulty.
- **Accommodation safety is not a principal contribution of any reviewed work.** Verified and filterable information about CCTV coverage, security staff and women-only policy for hotels and paying-guest stays, which a traveller needs before she arrives, lies outside the stated scope of the surveyed publications.
- **Hotspot prediction is presented as an analytical technique.** The hotspot studies produce crime-risk estimates, but the summaries do not describe feeding that estimate into the route a traveller is shown beside the fastest option, together with the time of day.
- **Some solutions depend on specific hardware or a native platform.** The ESP32 camera-based application and the native Android application each assume a particular device or operating system, whereas a web-based design built on widely available open technology can be reached from any browser.

SafeHer is positioned in exactly this gap. It combines a route-risk score, an administrator-verified accommodation directory, multi-path SOS with live location sharing and a community

rating layer in one authenticated web platform, and it keeps the crime-zone heatmap and the graph-based hotspot approach as an extension path.

A further point that emerges from the review is that most of the works are evaluated by their authors on small or prototype data sets, and few report results from use by travellers in real conditions. SafeHer is subject to the same limit at mini-project scale, and this report therefore presents its results as evidence of a working design and not as proof of effect in the field.

3.13 Lessons Carried into the Design

Each of the reviewed works left a specific mark on the design. They are collected here so that the link between the literature and the system is easy to follow.

- **Keep routing and scoring separate:** the Routes API documentation and SafeRoutes both suggest retrieving candidate routes from a routing service and scoring them independently, which is how the Trip and Route module is organised.
- **Treat time as an input, not an overlay:** the hotspot studies show that risk varies through the day, so the time of day is an input to every score rather than a fixed map layer.
- **Allow more than one kind of recipient:** Safety for HER motivated an alert design that is not limited to a fixed list of contacts.
- **Do not rely on a single trigger:** the voice-activated and shake-based approaches show that a traveller may not be able to reach her phone, so tap, voice and wearable triggers are all supported.
- **Make delivery reliable and visible:** the work on alert reliability under poor connectivity led to retried dispatch and a stored delivery status for every recipient.
- **Keep registration short:** the tracker application's emphasis on low-friction onboarding led to a three-step registration with emergency contacts added later.
- **Leave room for evidence and for better models:** the camera-based work and the graph-based hotspot work are both reflected in schema and design choices that allow them to be added later.

CHAPTER 4
DIAGRAMS

This chapter presents the structural and behavioural models of the SafeHer system.

4.1 Data Flow Diagrams

4.1.1 DFD Level 0 — Context Diagram

The context diagram establishes the system boundary, treating SafeHer as a single process exchanging requests and responses with two external entities, the User and the Admin.

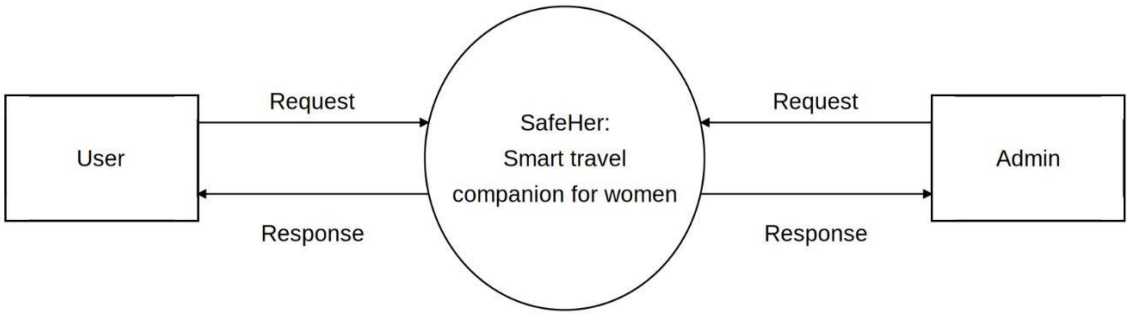


Figure 4.1.1: Data flow diagram — Level 0 (context diagram)

4.1.2 DFD Level 1 — Admin

The first level of decomposition on the admin side splits the single process into six admin functions: manage users, manage hotels/PGs, review management, SOS alert management, view reports and analytics, and chatbot content management, each reading from and writing to its own data store.

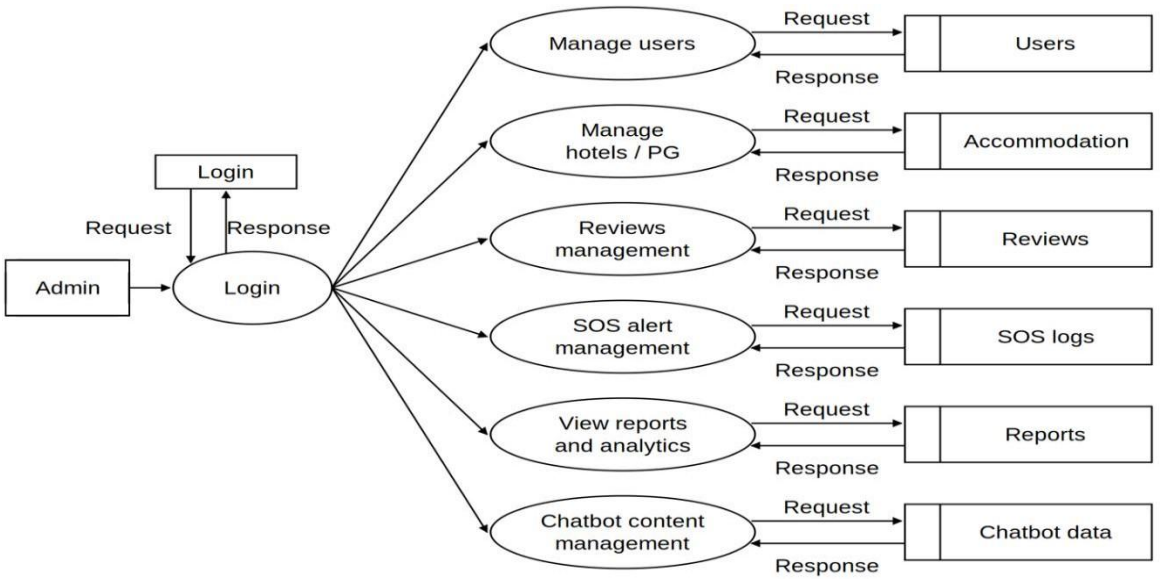


Figure 4.1.2: Data flow diagram — Level 1 (Admin)

4.1.3 DFD Level 1 — User

On the user side, the single process decomposes into profile and contact management, SOS management, search hotels/PGs, route and navigation, manage reviews/ratings, and chatbot and safety tips, mirroring the six core modules described in Chapter 5.

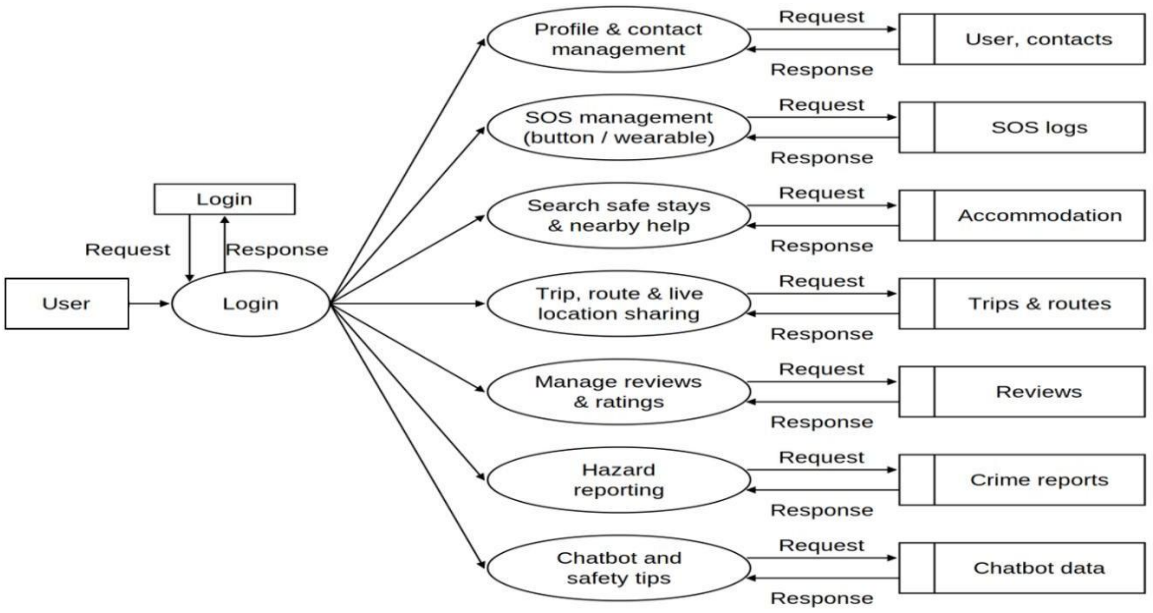


Figure 4.1.3: Data flow diagram — Level 1 (User)

4.1.4 Data Stores

The two Level 1 diagrams read from and write to a small number of data stores. Table 4.2 lists them with the processes that use each store, which also shows where the administrator and the traveller share data and where they do not.

Table 4.2: Data stores used in the Level 1 diagrams

Data store	Used by	Content
Users / contacts	User management (Admin); profile and contact management (User)	Accounts, profiles and the emergency contacts registered by each user.
Accommodation	Hotel and PG management (Admin); safe-stay search (User)	Hotel and paying-guest records with their verification status and safety amenities.
Reviews	Review management (Admin); reviews and ratings (User)	Ratings and comments attached to places.
SOS logs	SOS alert management (Admin); SOS management (User)	The SOS alerts raised, their recipients and their outcome.
Trips and routes	Trip, route and live location sharing (User)	Planned trips and the scored candidate routes.
Crime reports	Reports and analytics (Admin); hazard reporting (User)	Hazard and crime reports that feed analytics and the crime-zone heatmap.
Chatbot data	Chatbot content management (Admin); chatbot and safety tips (User)	Safety content and queries handled by the chatbot.

4.2 Use Case Diagram

The use case diagram identifies the two actors, User and Admin, and the services available to each. Every use case other than registration, login and logout requires an authenticated session, which is enforced at the API layer rather than by hiding options in the interface.



Figure 4.2: Use case diagram of SafeHer

4.3 Class Diagram

The class diagram shows the persistent domain entities and the operations defined on each. The User entity is central, with one-to-many relationships to EmergencyContact, Trip and SafePlace-related content, and a triggers relationship into SOSAlert.

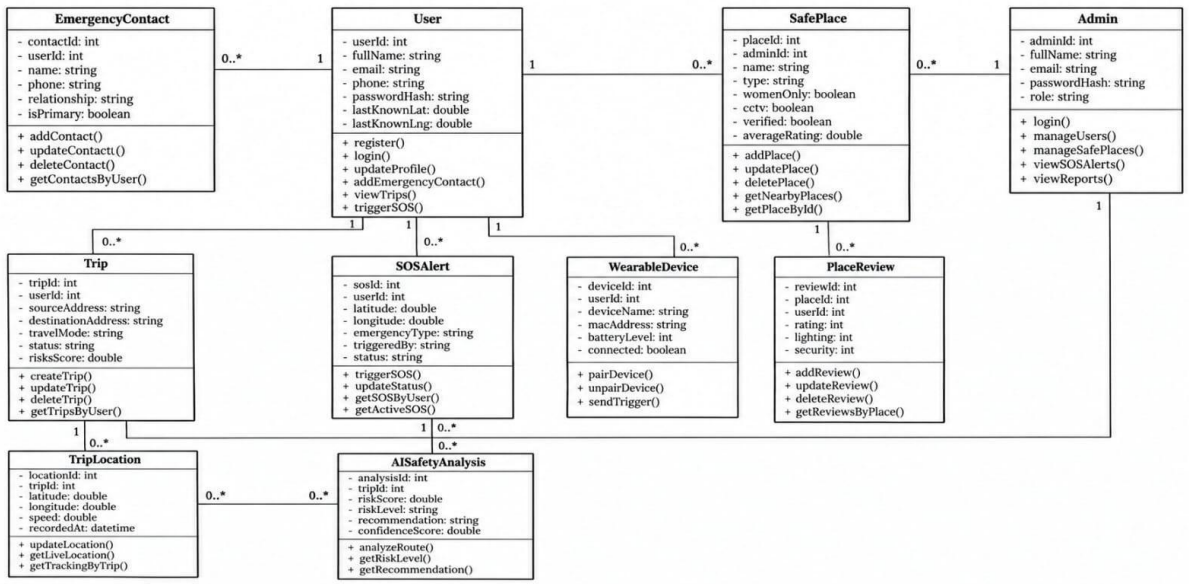


Figure 4.3: Class diagram of domain entities

4.4 State Diagram

The state diagram traces a user's journey through the platform, from the splash/welcome state through login, the home dashboard, and into the five functional states — Trip and Route, Safe Places, Emergency/SOS, Notifications, and Profile and Settings — with a dedicated Admin and AI module reachable for administrator users.

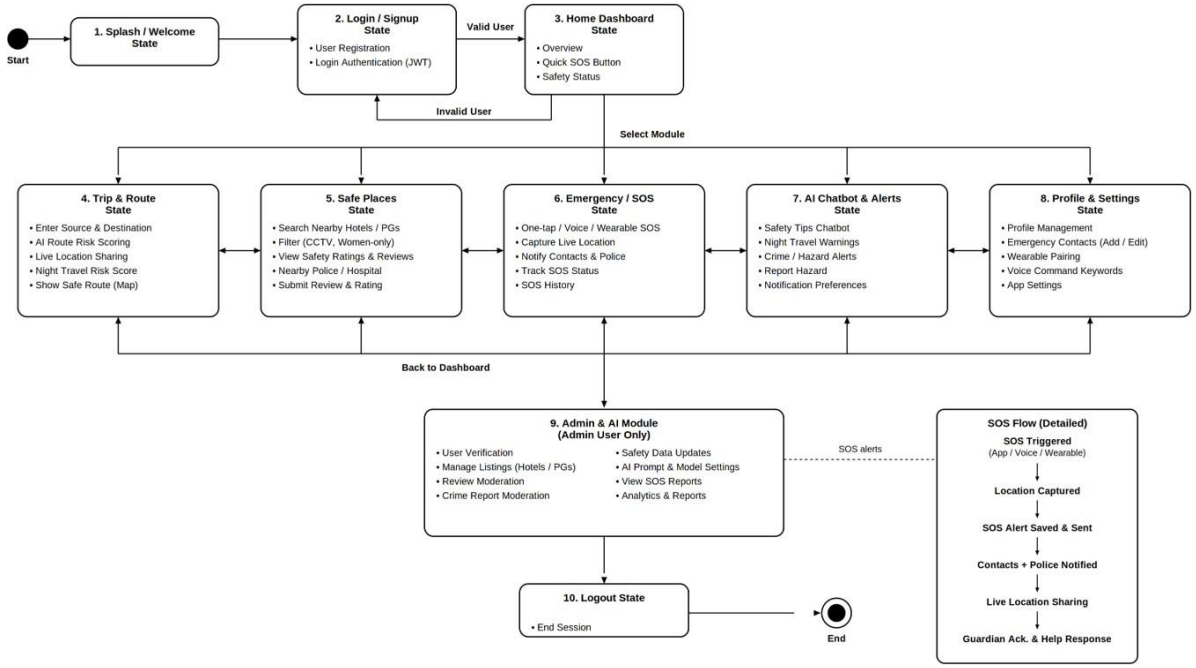


Figure 4.4: State diagram of the SafeHer lifecycle

4.5 Activity Diagram

The activity diagram shows the full flow of the SafeHer app from start to end. The user opens the web app and registers if new, then logs in, and an invalid login shows an error and returns to the login step. After a valid login the user reaches the dashboard and selects a feature. In

Trip & Route, the AI analyses the risk, shows a safe route and risk score, and the trip is tracked with live location sharing until it is complete. In Safe Places, the user searches verified places, views ratings and can submit a review. In Emergency SOS, the user triggers an alert by button, voice or wearable and has 5 seconds to cancel it. If it is not cancelled, the system saves the location, notifies the contacts and alerts the admin and police at the same time, and the SOS is marked resolved once help arrives. In the AI Chatbot, the user asks a safety question and gets advice. The Admin module covers login, managing users and listings, moderating reports, monitoring SOS alerts and viewing reports. After any feature the user can pick another one or log out, which ends the flow.

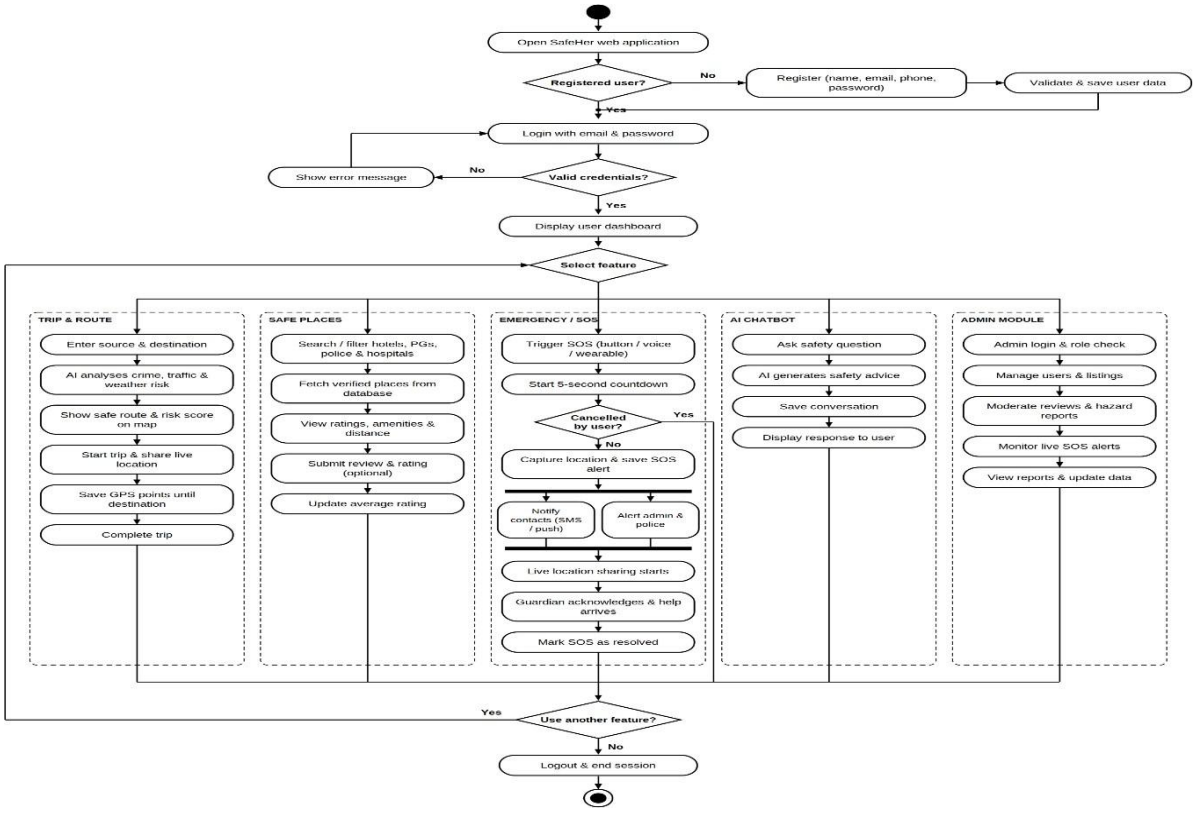


Figure 4.5: Sequence diagram of a single SOS request

4.6 Sequence Diagram

The sequence diagram shows how the User, Admin, System, AI and Database interact over time in SafeHer. A user logs in, and the system checks the details in the database before confirming the login. For a trip, the system asks the AI to analyse the route risk and shows the user a safe route, saving the trip in the database. When searching for safe places, the system fetches verified places from the database and shows them with ratings, and the user can submit a review. In an emergency, the user triggers an SOS, the system saves it with the live location and alerts the admin and emergency contacts. The user can also ask the AI chatbot safety

questions, and the system saves the conversation. Finally, the admin logs in to manage users and listings and to monitor SOS alerts, and the system updates the database and shows reports.

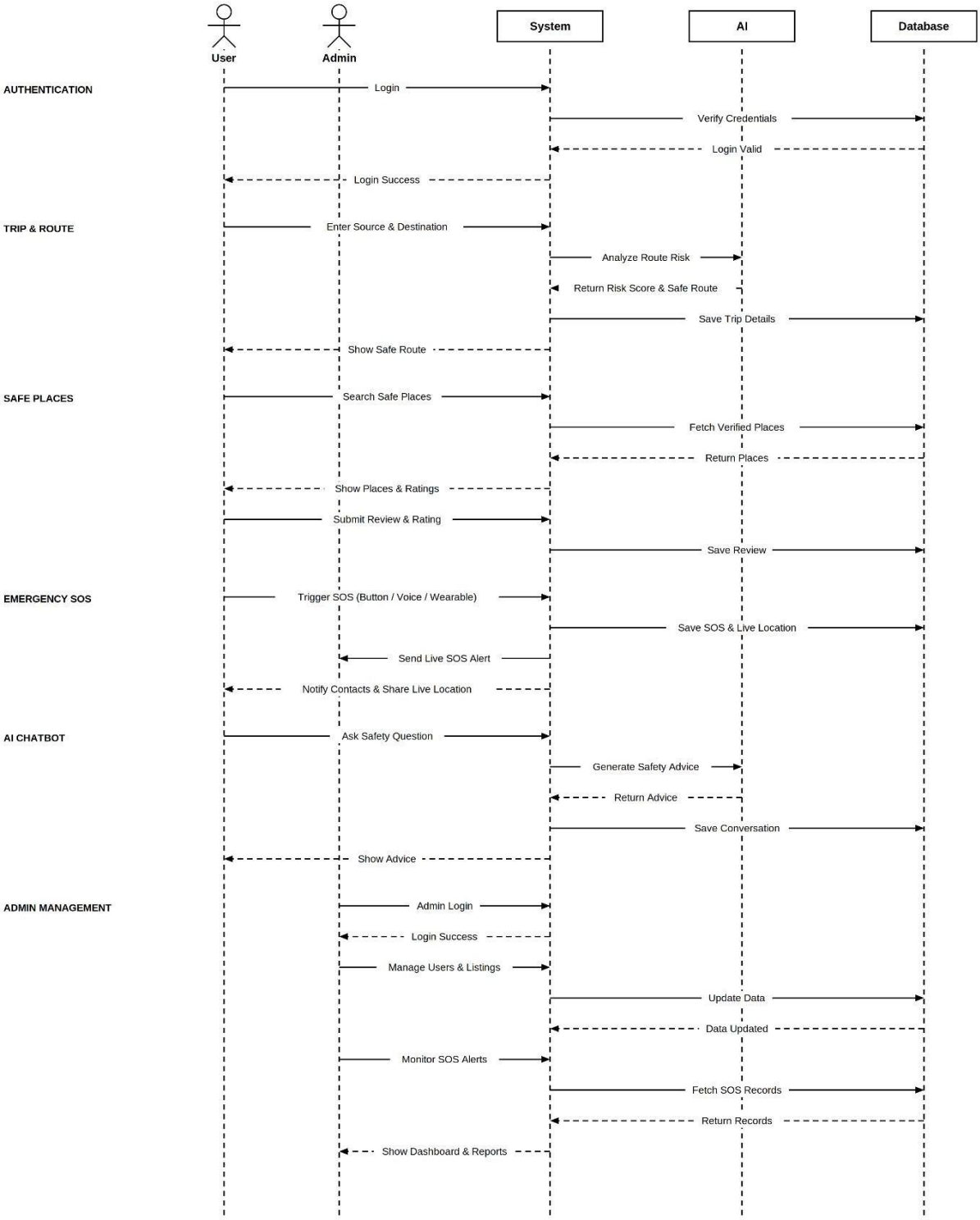


Figure 4.6 Sequence Diagram

CHAPTER 5
METHODOLOGY

5.1 Module Descriptions

SafeHer is organised into six core modules on the client and a matching set of route handlers and services on the Node.js/Express backend, each backed by its own tables in the MySQL schema.

Table 5.1: Summary of implemented modules and responsibilities

No.	Module	Responsibility
1	Authentication and Profile	Handles registration, JWT-protected login with refresh tokens, and the user profile.
2	Trip and Route	Takes a source and destination, fetches candidate routes from the Google Maps Routes API, scores each one with the route-risk model and shows the safer route beside the fastest.
3	Safe Places	Provides the verified hotel and PG directory, filtered by verification, CCTV, women-only status and rating, with photographs and community reviews.
4	Emergency SOS & Wearable Integration	Raises SOS by tap, voice or wearable, opens a live-tracking session and notifies emergency contacts and, where integrated, the police.
5	Notifications	Pushes weather, traffic and night-travel warnings and safety tips through Firebase Cloud Messaging and returns SOS delivery confirmations.
6	Admin and AI	Gives administrators a console to verify places, moderate posts, track live SOS alerts and view analytics, with AI-assisted risk and chatbot support.

5.1.1 Authentication and profile module

The registration screen collects the user's full name, email address, phone number and a password with confirmation, and creates a User record together with a linked Profile. All subsequent routes are protected by a JWT authentication middleware, so a request without a valid token is rejected with an authentication error rather than silently returning empty data. A refresh-token table allows a session to be renewed without forcing the user to log in again on every visit.

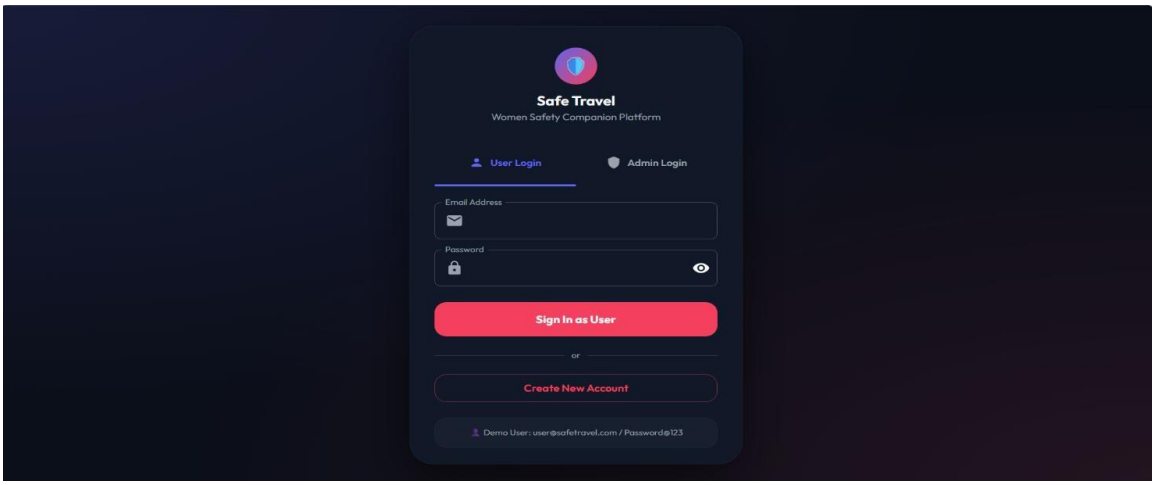


Figure 5.1.1: User login

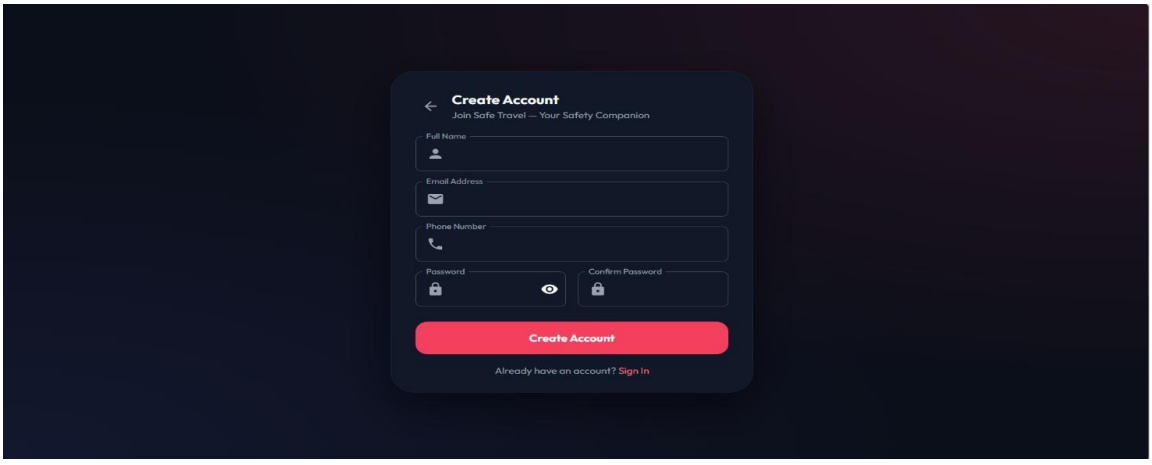


Figure 5.1.2: User Registration

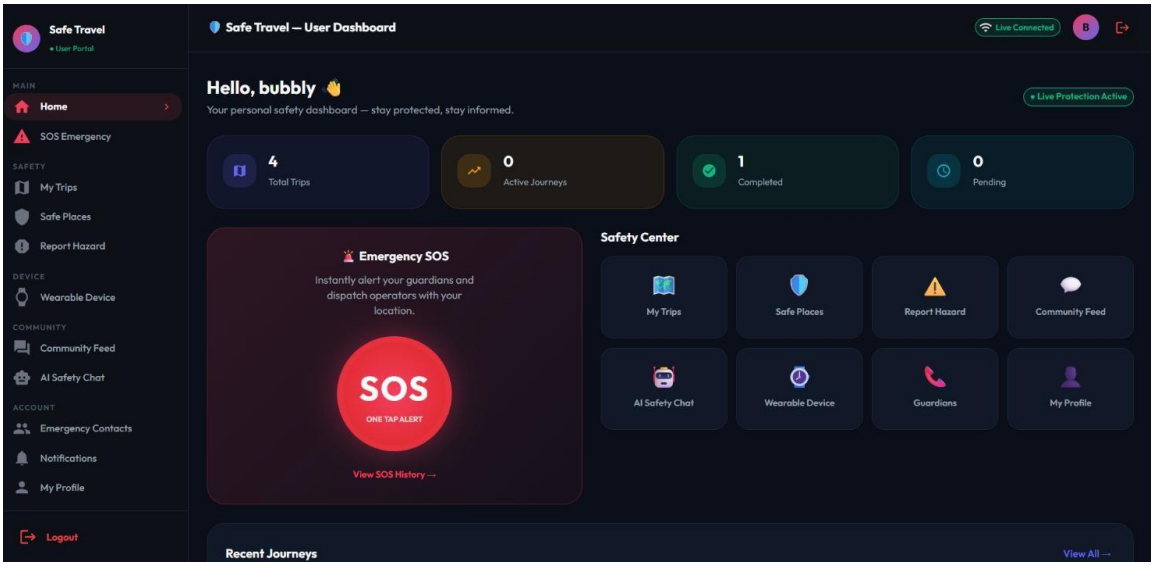


Figure 5.1.3: User Dashboard

5.1.2 Trip and route module

When a user enters a source and destination, the backend requests one or more candidate routes from the Google Maps Routes API, then re-scores each candidate using the route-risk model described in section 5.2, combining historical crime density along the route corridor, the

current time of day, and, where available, lighting and crowd-density indicators for that corridor. The safer route is presented to the user alongside the fastest one, with the underlying risk score shown so the choice remains the traveller's own.

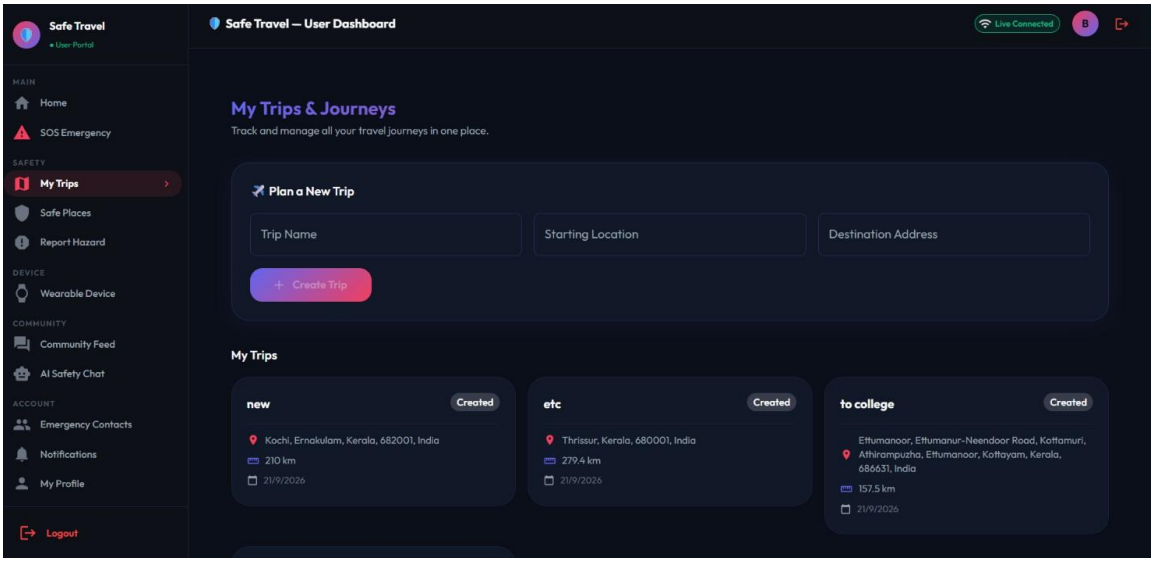


Figure 5.1.4: Trip & Journeys

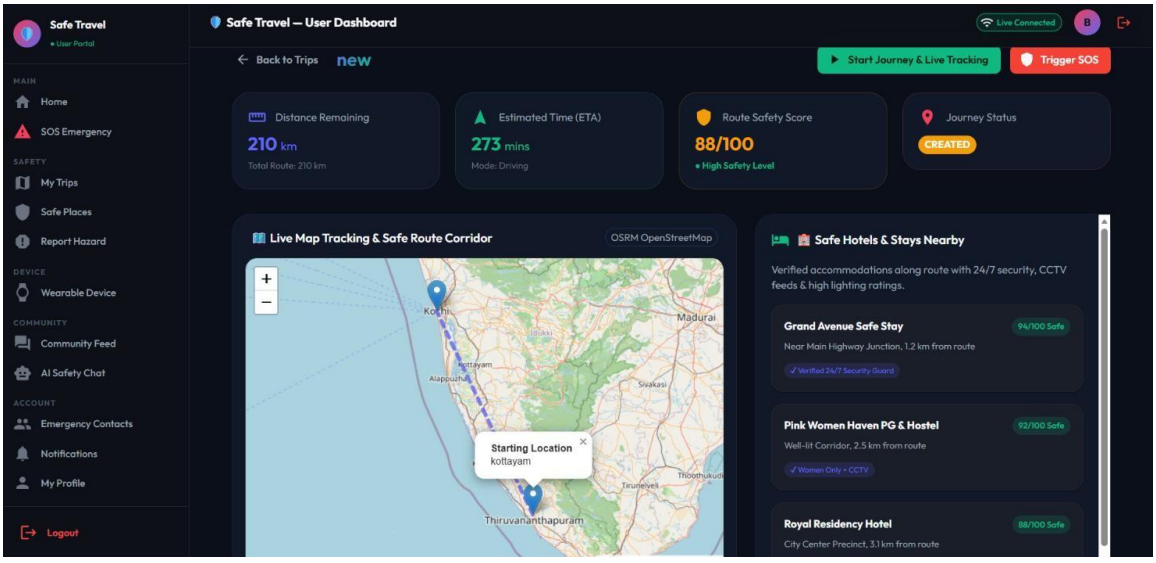


Figure 5.1.5: Route Predictions

5.1.3 Safe places module

The SafePlace entity stores a name, category, verification flag and aggregate rating for each hotel or PG; PlaceImage and PlaceReview entities attach photographs and user reviews. A search request can be filtered by verified status, CCTV, women-only status and minimum rating, and only listings an administrator has marked verified are shown with the verified badge.

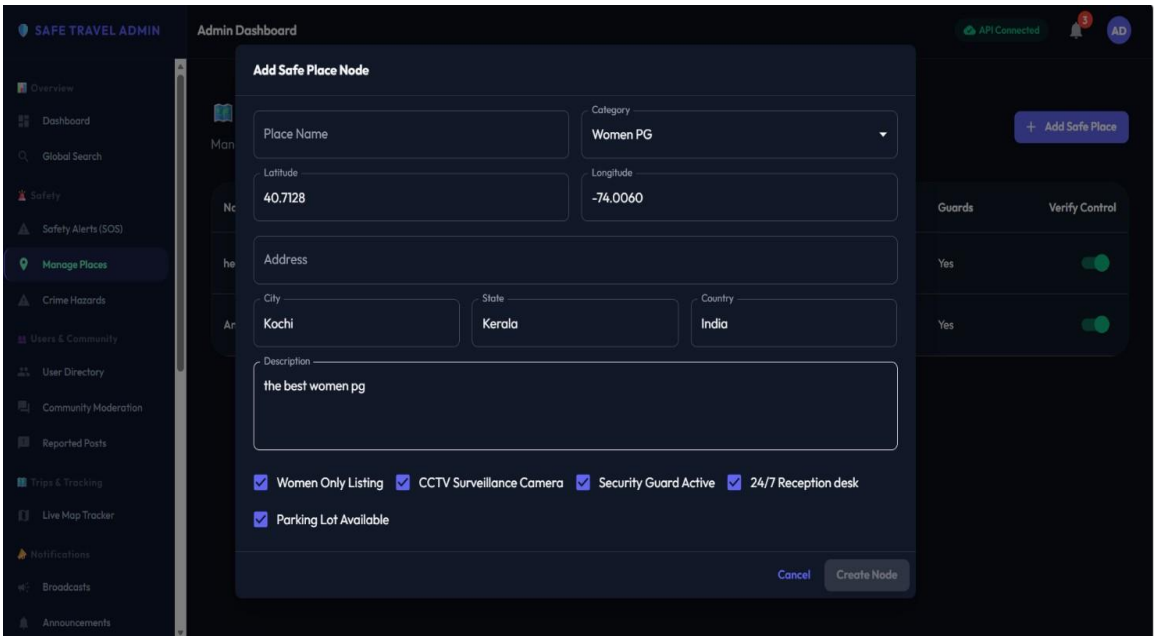


Figure 5.1.6: Safe Place

5.1.4 Emergency SOS & Wearable Integration Module

Triggering SOS, by tap, by voice command, or from a paired wearable device, creates an SOSAlert record carrying the user's current location and a status field, and fans that alert out to every configured SOSRecipient (emergency contacts and, where integrated, police) through the notification service. A LiveTrackingSession is opened at the same moment so that a recipient who accepts the alert can watch the user's location update in real time until the session is ended.

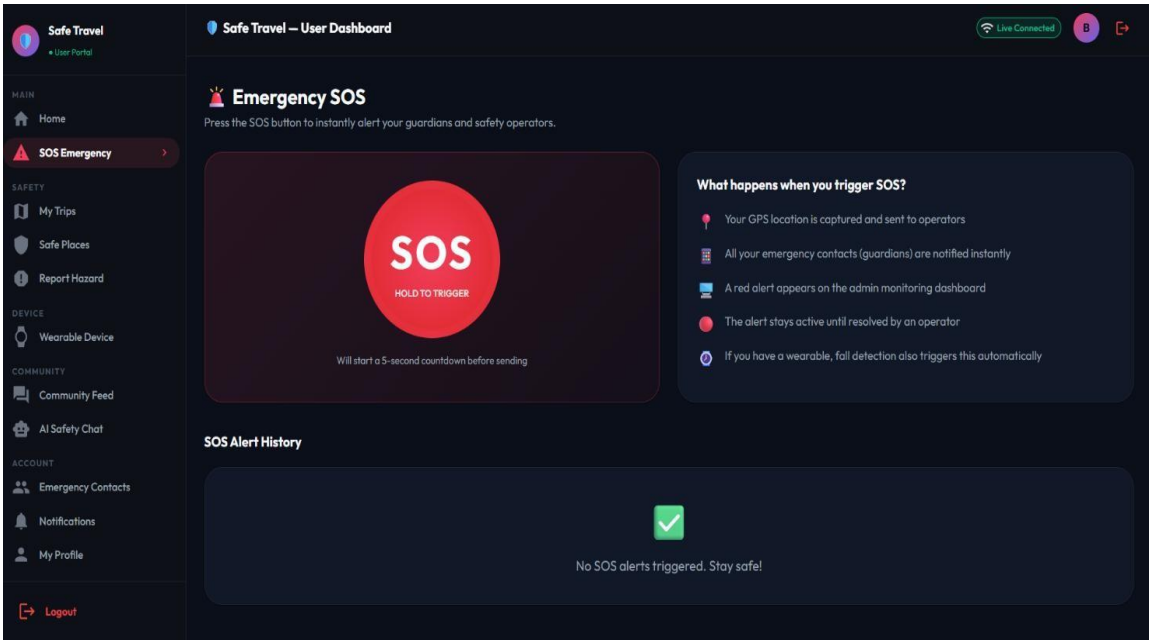


Figure 5.1.7: SOS alert

5.1.5 Notifications module

The notification service pushes weather, traffic and night-travel warnings, together with safety tips, through Firebase Cloud Messaging, and exposes a preference screen so a user can tune which categories of alert she receives. The same service channel carries SOS delivery confirmations back to the alerting user.

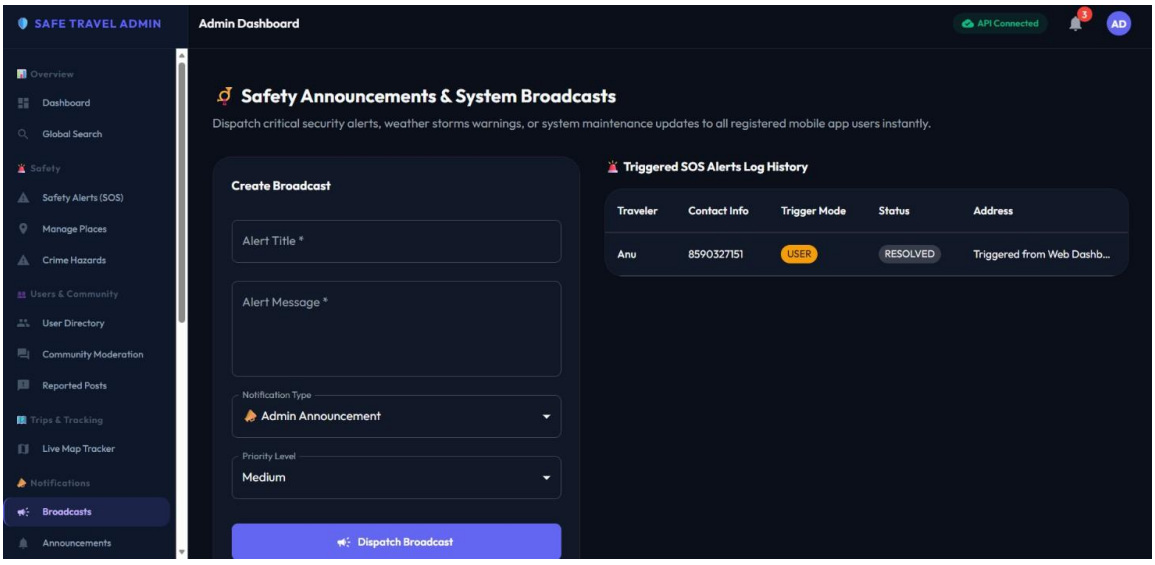


Figure 5.1.8: Notifications

5.1.6 Admin and AI module

An administrator signs in through a separate operator console (see Figure 5.1.9) and lands on a dashboard summarising total users, trips today, SOS alerts today, safe places, community posts, crime reports, notifications sent and AI requests (see Figure 5.1.10). From this console an administrator verifies or rejects a place listing, moderates a reported community post, reviews the status of active SOS alerts on a live map tracker, and reviews platform-wide analytics.

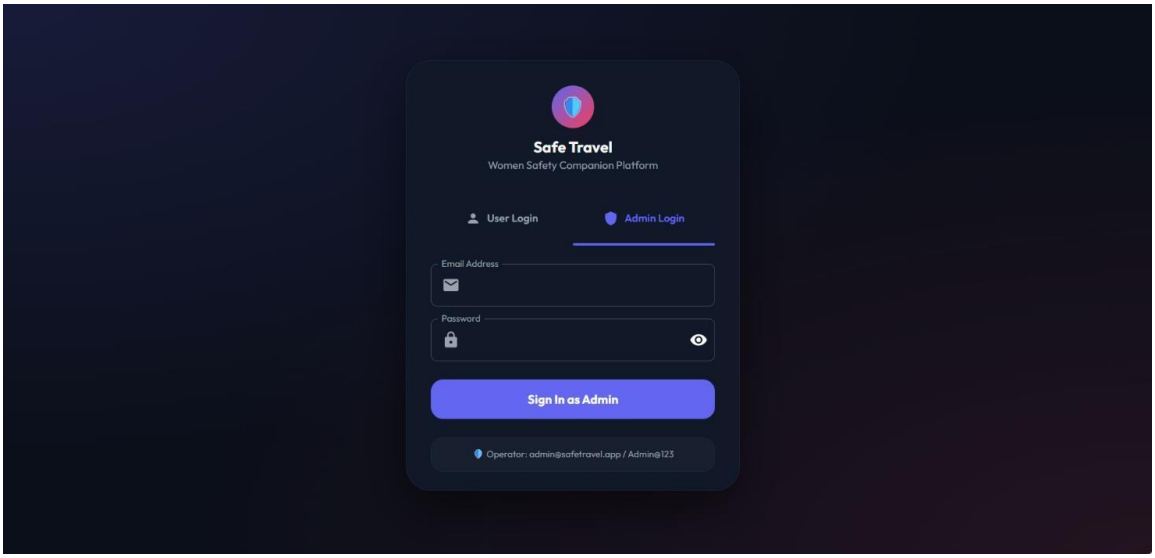


Figure 5.1.9: Admin login interface

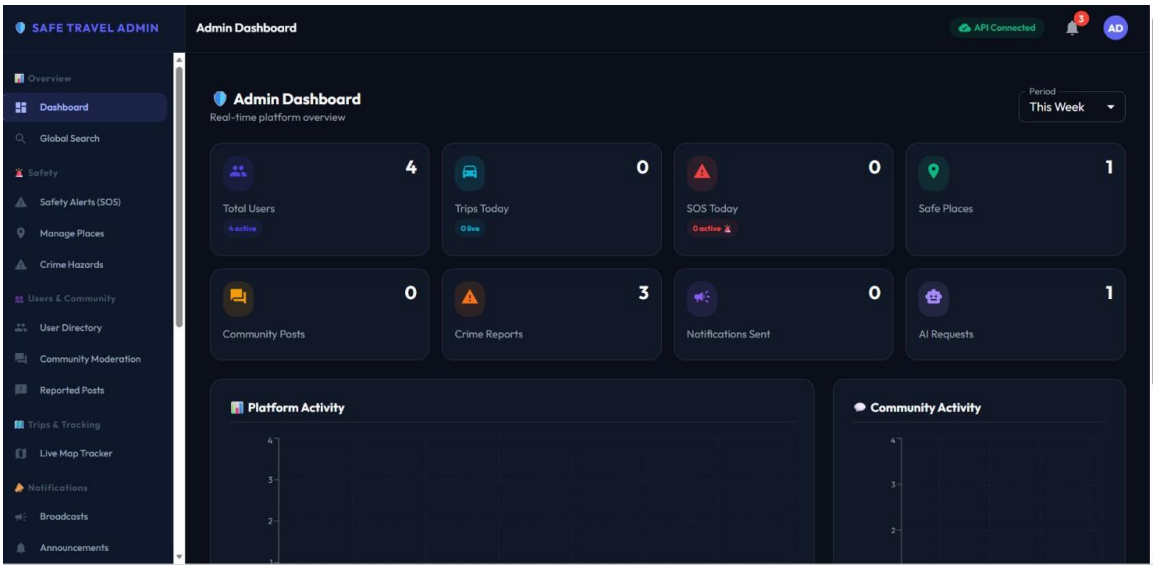


Figure 5.1.10: Admin dashboard

5.2 Algorithms

5.2.1 Route-risk scoring algorithm

The route-risk score is the central algorithm in SafeHer because it converts several independent safety signals into one number the traveller can act on. Each candidate route returned by the Routes API is divided into short corridor segments; each segment is scored against the four signals below and the segment scores are combined into a single route-level score.

Table 5.2: Route-risk signal weights used in the scoring module

Signal	Description	Weigh t	Rationale
Historical crime density	Reported incidents per unit length of the corridor, normalised over the last 12 months	0.40	Strongest available indicator of realised risk on a route
Time-of-day factor	Elevated weighting applied between 8 PM and 6 AM, derived from the hotspot literature reviewed in Chapter 3	0.25	Risk is not uniform across the day even on the same street
Lighting	Street-lighting presence sourced from map metadata and community reports	0.20	Poor lighting is consistently cited as a deterrent to feeling safe
Crowd density	Typical footfall for the corridor and time slot, sourced from places data and community reports	0.15	Isolated stretches carry materially higher risk than busy ones

PSEUDOCODE — Route risk scoring

```
ALGORITHM: score_route(route, time_of_day)
INPUT : route (list of corridor segments), time_of_day
OUTPUT : route_risk_score in [0, 1] (0 = safest, 1 = highest risk)
1. total <- 0 ; total_length <- 0
```

```
2. FOR each segment s IN route DO
    3. crime <- normalised_crime_density(s)
    4. time_factor<- 1.4 IF time_of_day IN [20:00, 06:00) ELSE 1.0
    5. lighting <- lighting_score(s) // 0 = well lit, 1 = unlit
    6. crowd <- 1 - crowd_density_score(s) // low crowd -> higher risk
    7. seg_score <- (0.40*crime + 0.20*lighting + 0.15*crowd) *
time_factor
    8. total <- total + seg_score * length(s)
    9. total_length <- total_length + length(s)
10. route_risk_score <- clip(total / total_length, 0, 1)
11. RETURN route_risk_score
```

5.2.2 SOS dispatch and live-tracking algorithm

```
ALGORITHM: trigger_sos(user, trigger_source, location)
INPUT : user, trigger_source (tap | voice | wearable), location
OUTPUT : sos_alert record, live_tracking_session
1. alert <- create SOSAlert(userId=user.id, location=location,
status='ACTIVE', alertTime=now())
2. session <- create LiveTrackingSession(userId=user.id, status='ACTIVE')
3. recipients <- user.emergencyContacts + (policeIntegration IF enabled)
4. FOR each r IN recipients DO
    5. create SOSRecipient(sosId=alert.id, contactId=r.id,
notificationStatus='SENT')
    6. dispatch_notification(r, alert, session.viewLink)
    7. IF dispatch fails THEN retry up to 3 times with backoff
8. RETURN (alert, session)
```

The guard in step 7 is deliberate: unlike a route-risk mis-score, a failed SOS delivery has an immediate real-world cost, so the dispatch step retries with backoff rather than failing silently, and the delivery status of every recipient is recorded against the alert for later review.

5.2.3 Worked Example of Route-Risk Scoring

To show how the algorithm of section 5.2.1 turns raw signals into a score, consider two candidate routes between the same source and destination at 9 PM. All values are illustrative and are chosen only to demonstrate the calculation; they are not drawn from a real dataset.

Route A is the fastest route and consists of two segments. The first is 400 m long with a normalised crime density of 0.30, a lighting score of 0.20 and a crowd-risk value of 0.30. The second is 600 m long with a crime density of 0.60, a lighting score of 0.70 and a crowd-risk value of 0.80. Because the journey is at night, the time factor is 1.4.

The score of the first segment is $(0.40 \times 0.30 + 0.20 \times 0.20 + 0.15 \times 0.30) \times 1.4 = 0.205 \times 1.4 = 0.287$, and that of the second is $(0.40 \times 0.60 + 0.20 \times 0.70 + 0.15 \times 0.80) \times 1.4 = 0.500 \times 1.4 = 0.700$. Weighting each by its length gives a route score of $(0.287 \times 400 + 0.700 \times 600) / 1000 = 0.535$.

Route B is 200 m longer and also has two segments: 500 m with signal values of 0.20, 0.15 and 0.20, and 700 m with values of 0.30, 0.25 and 0.30. Their scores are $0.140 \times 1.4 = 0.196$

and 0.215 × 1.4 = 0.301, which gives a route score of (0.196 × 500 + 0.301 × 700) / 1200 = 0.257.

The traveller is therefore shown Route B first at night, with a score of 0.257 against 0.535, even though it is somewhat longer. If the same two routes were scored at midday the time factor would be 1.0 and the scores would fall to 0.184 for Route B and 0.382 for Route A, so the safer route would still be preferred but by a smaller margin, which is the behaviour exercised in test case TC03. The example also shows why the score is explainable: each segment's contribution can be traced back to the three signals and displayed beside the route.

5.3 Architecture

SafeHer follows a four-layer architecture with a clear dependency direction: the presentation layer depends only on the application layer beneath it, which depends only on the service layer, which depends only on the data layer.

- Presentation layer: React.js single-page application, consuming the REST API and a WebSocket channel (Socket.IO) for live-tracking updates.
- Application layer: Express.js route handlers and controllers across the User & Auth, Trip & Route, Safe Places, Emergency, Notifications and Admin & AI areas.
- Service layer: the route-risk scoring service, the SOS dispatch service, the notification service, and the Google Maps Platform integration.
- Data layer: the MySQL relational database described by the ER diagram in Figure 4.8, plus a media store for place images and profile pictures.

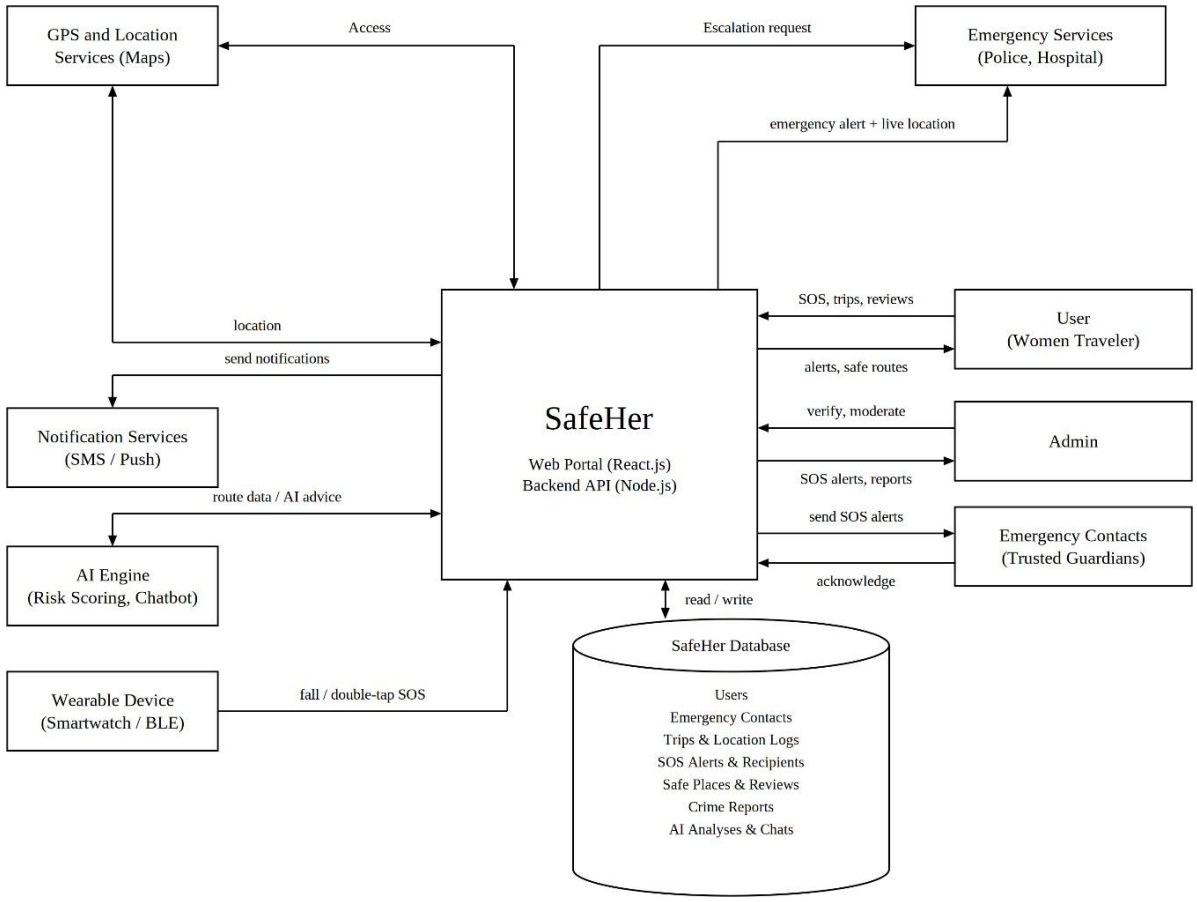


Figure 5.3.1: System architecture of SafeHer

5.4 Case Study

To make the pipeline concrete, this section traces a single journey end to end. A user, Sarah, opens the Plan Safe Journey option on her dashboard (Figure 5.1.3) and enters a destination for a 9 PM trip. The Trip and Route service requests three candidate routes from the Google Maps Routes API, then scores each using the algorithm in section 5.2.1; the 8 PM–6 AM time factor raises the weight given to lighting and crowd density for this request. The route with the lowest risk score is presented first, with its score and the individual signal contributions shown so Sarah can see why it was preferred.

Partway through the walk, Sarah feels uneasy and taps the SOS control. The Emergency module creates an SOSAlert carrying her current GPS coordinates, opens a LiveTrackingSession, and dispatches a notification to her three registered emergency contacts, one of whom accepts the live-tracking link and begins watching her position update in real time until Sarah marks herself safe, at which point the session and the alert are both closed and archived to her private SOS history.

5.5 Technology Stack

SafeHer is built entirely from open, well-supported technologies, which keeps licensing cost negligible and allows the platform to run in any modern browser without installation. Table 5.3 lists each technology and the role it plays.

Table 5.3: Technologies used and their roles

Layer	Technology	Role in SafeHer
Presentation	React.js 18.x	Single-page application for the traveller interface and the administrator console.
Application	Node.js 18 LTS, Express.js 4.x	REST API, route handlers and controllers for the six modules.
Data	MySQL 8.0, Sequelize / Prisma	Relational storage of users, trips, SOS alerts, safe places and community content, accessed through an ORM.
Real-time	Socket.IO 4.x	Pushes live location updates from the travelling user to the recipients of an SOS alert.
Security	JWT, bcrypt	Token-based authentication, password hashing and role-based access to the admin console.
Notifications	Firebase Cloud Messaging, Twilio SMS	Push notifications, safety warnings and alert delivery to contacts.
Mapping	Google Maps Platform (Routes API, Places)	Candidate routes, geocoding and places data for the route-risk and safe-places modules.
Browser APIs	Geolocation API, Web Bluetooth API	Device location for live tracking and Bluetooth pairing of the wearable trigger.
Version control	Git	Management of the source code across development increments.

The technologies follow the four-layer structure described in section 5.3. The presentation layer depends only on the REST interface of the application layer, the application layer delegates the substantive logic to the service layer, and only the service layer reads and writes the data layer. External services such as the Google Maps Platform and the notification providers are reached from the service layer, so that any one of them can be replaced without changing the layers above it.

Choosing a web stack rather than a native one also means a single code base serves every device a traveller is likely to carry. The limitation of this choice, background location access on mobile browsers, is discussed in Chapter 9 and is the main reason native applications appear in the future scope.

5.6 Design Decisions and Rationale

Several design decisions shaped the platform, and the reasons for them are recorded here because they explain features that might otherwise appear arbitrary.

A web application rather than a native application. A web client runs on any phone or laptop without installation, can be updated for every user at once and keeps the project within the reach of a single developer. The price is limited background location access in mobile browsers, which is recorded as a limitation and addressed by the native applications proposed in Chapter 9.

A relational database rather than a document store. The data of SafeHer is heavily relational: a user has many contacts and trips, an alert has many recipients, and a place has many reviews and images. MySQL enforces these relationships through keys and constraints and supports the reporting queries needed by the administrator dashboard, so integrity is maintained by the database as well as by the application code.

A separate scoring service. The route-risk logic is kept apart from the request handlers so that it can be tested alone, changed without touching the rest of the application and, in future, replaced by a trained model behind the same interface.

Re-ranking rather than routing. SafeHer does not compute routes itself. It asks an established routing service for candidates and re-ranks them, which avoids reimplementing a hard problem and keeps the contribution of the project focused on the safety score.

WebSockets for live tracking. Pushing position updates through Socket.IO avoids the delay and overhead of repeated polling, which matters when a contact is watching a user's position during an emergency.

Explainable scoring over an opaque model. A weighted combination of named signals is simple enough that the traveller, the administrator and the examiner can all see why one route scored better than another. At the data volumes available to a mini project, this transparency is worth more than the marginal accuracy an opaque model might offer.

Role-based access with an audit trail. Administrator functions are protected by a role check and every state-changing action is logged, so that the power given to an administrator is matched by accountability for how it is used.

5.7 Security Architecture

Because SafeHer handles personal and location data, security was treated as a design concern from the outset rather than added at the end. The measures can be described by concern.

Identity and authentication. Passwords are hashed with bcrypt before storage and are never kept in readable form. Sessions use short-lived JSON Web Tokens paired with longer-lived refresh tokens held in the RefreshToken table, which allows a session to be revoked without waiting for a token to expire.

Authorisation. Every protected route passes through middleware that validates the token and, for operator functions, checks the role. A standard user token presented to an administrator endpoint is rejected, as confirmed by the security testing described in Chapter 6.

Data protection. Contact details, trip history and SOS history are scoped to the owning account, so one user cannot retrieve another's records. A tracking link is tied to a live session and ceases to be useful once the session closes.

Transport and configuration. In a production deployment all traffic should be carried over HTTPS, and secrets such as database credentials and API keys should be held in environment configuration rather than in source code. The delivered development configuration should be hardened in this way, as noted in section 7.2.

Input handling. Requests are validated before they reach the service layer, and access to the database through an ORM that issues parameterised queries reduces the risk of injection.

Accountability. Administrative actions are written to the audit log, and the delivery status of an SOS alert is stored for every recipient, so that the history of any alert can be reconstructed.

These measures do not make the platform immune to attack, and the report does not claim that they do. They do ensure that the most important boundaries, between users, between users and administrators, and between the platform and the outside world, are explicit and testable.

CHAPTER 6
IMPLEMENTATION DETAILS

6.1 Implementation Procedures

6.1.1 Development environment

Development was carried out using Node.js 18 LTS on the backend and React 18 on the frontend, with MySQL 8 as the relational database. A single environment configuration file holds database credentials, the JWT signing secret, the Google Maps API key and the Firebase Cloud Messaging credentials, keeping secrets out of source control.

6.1.2 Project scaffolding and data model

The backend is organised as an Express.js application with a route/controller/service layering matching the six modules in Chapter 5, and the twelve-plus entity relational schema shown in Figure 4.6 was implemented through migration scripts so that the schema definition remains a single, reviewable source of truth.

6.1.3 Authentication and security implementation

Passwords are hashed with bcrypt before storage, sessions are managed through short-lived JWT access tokens paired with longer-lived refresh tokens stored in the RefreshToken table, and every protected route is guarded by an authentication middleware. Role-based checks distinguish an ordinary user from an administrator, so the admin console's verification and moderation endpoints are unreachable from a standard user session.

6.1.4 Route-risk service implementation

The risk-scoring service is implemented as an independent Node.js service exposed through an internal REST endpoint, following the same separation-of-concerns principle observed in the SafeRoutes literature reviewed in Chapter 3, so that the scoring logic can be retrained or replaced without touching the route handlers that call it.

6.1.5 SOS and live-tracking implementation

Socket.IO is used to push live location updates from the travelling user's client to any connected LiveTrackingViewer session, avoiding the latency of repeated HTTP polling. Notification delivery to emergency contacts uses Firebase Cloud Messaging for push notifications and falls back to SMS for a contact who has not installed the companion notification channel.

6.1.6 Admin console implementation

The admin console is a separate authenticated area of the same React application, gated by the administrator role check described above, and every state-changing action taken from it,

verifying a place, resolving a reported post, closing an SOS alert, is written to the AuditLogs table with the acting admin, the affected user, the action and a timestamp.

6.1.7 Database schema implementation

Table 6.1: Application database tables and key columns

Table	Key columns	Relationship
User	userId, fullName, email, phone, password, createdAt	Parent of EmergencyContact, Trip, SOSAlert
EmergencyContact	contactId, userId (FK), name, phone	Many-to-one with User
Trip	tripId, userId (FK), status, riskScore, startTime	Many-to-one with User; has many TripLocation
SOSAlert	sosId, userId (FK), location, status, alertTime	Triggers from User; has many SOSRecipient
SOSRecipient	recipientId, sosId (FK), contactId (FK), notificationStatus	Many-to-one with SOSAlert
SafePlace	placeId, name, category, verified, rating	Has many PlaceImage, PlaceReview
CommunityPost	postId, userId (FK), title, content, visibility	Has many PostComment, PostLike
LiveTrackingSession	sessionId, userId (FK), tripId (FK), status	Part of Trip; viewed by LiveTrackingViewer
AdminUsers / AuditLogs	adminId, name, role / auditId, adminId (FK), action	AdminUsers performs AuditLogs

6.1.8 Source organisation

```
safeher/
  client/                  React.js frontend
    src/pages/               Login, Register, Dashboard, Trip, SafePlaces,
SOS
    src/components/          Map view, SOS button, review card, chat
widget
  server/                  Node.js / Express backend
    routes/                  auth, trips, places, sos, community, admin
    controllers/             thin request handlers
    services/                routeRiskService, sosService,
notificationService
    models/                  Sequelize/Prisma models matching Fig. 4.6
    middleware/              auth.js, roleCheck.js
  admin-console/           operator dashboard views
  migrations/              database schema migrations
  .env.example             configuration template
```

6.2 Testing Methods and Results

6.2.1 Testing strategy

- Unit testing of the route-risk scoring function and the SOS dispatch function in isolation.

- Integration testing of the registration, login, trip-planning and SOS-trigger flows end to end against a test database.
- System testing across all six modules, covering both the user-facing application and the admin console.
- Security testing of authentication boundaries, confirming that admin-only endpoints reject a standard user token.

Table 6.2: Unit and integration test suite results

Test class	Test case	Expected behaviour	Result
RouteRiskTests	test_high_crime_segment_scores_high	A segment with elevated historical incident density returns a score above 0.6	Pass
RouteRiskTests	test_night_time_factor_applied	Identical segment scores higher between 20:00 and 06:00 than at midday	Pass
SOSDispatchTests	test_alert_creates_recipients_for_all_contacts	Every registered emergency contact receives an SOSRecipient row	Pass
SOSDispatchTests	test_failed_delivery_retries	A simulated delivery failure is retried up to three times	Pass
AuthTests	test_admin_route_rejects_user_token	A standard user JWT is rejected on an admin-only endpoint	Pass
AuthTests	test_password_is_hashed	The stored password value is never the plaintext password	Pass
PlacesTests	test_unverified_place_excluded_from_default_search	A place pending verification is hidden unless explicitly requested	Pass
IntegrationTests	test_full_registration_to_first_trip_flow	A new user can register, add a contact and plan a scored trip	Pass

Table 6.3: Representative system test cases

ID	Scenario	Expected outcome	Result
TC01	Register with a valid email and strong password	Account created, profile initialised	Pass
TC02	Register with a duplicate email	Rejected with a clear validation message	Pass
TC03	Plan a route at 9 PM vs 9 AM for the same source/destination	9 PM route weighted more heavily toward the lower-risk option	Pass
TC04	Search safe places filtered by CCTV and women-only	Only matching verified listings are returned	Pass

ID	Scenario	Expected outcome	Result
TC05	Trigger SOS with no emergency contacts configured	Clear warning shown; alert still logged for the user's own history	Pass
TC06	Trigger SOS with three contacts configured	All three receive a live-tracking link within the expected latency	Pass
TC07	Admin verifies a pending place listing	Listing becomes visible with the verified badge to all users	Pass
TC08	Admin resolves a reported community post	Post is hidden from public view and logged in AuditLogs	Pass
TC09	Unauthenticated access to the dashboard route	Redirected to the login screen	Pass
TC10	Wearable-triggered SOS (simulated BLE event)	Same SOSAlert flow triggered as the in-app SOS button	Pass

All listed cases passed. In TC-05, the system was deliberately designed not to block the alert when no emergency contact is configured, since recording the incident to the user's own history remains useful even when no recipient could be notified.

6.3 Organisation of the Database Schema

The tables of Table 6.1 and the entity relationship diagram of Figure 4.6 can be read as seven groups, each serving one part of the platform. Grouping them this way makes it easier to see which module owns which data, and which entities are shared across modules. Table 6.4 lists the groups.

Table 6.4: Entity groups in the database schema

Group	Entities	Purpose
User and access control	User, Roles, Permissions	Accounts, and what each kind of account is permitted to do.
Safety content	SafePlace, PlaceReview, PlaceImage	Verified accommodation with its reviews and photographs.
Community	CommunityPost, PostComment, PostLike, SavedPost	Content shared within the women-only community.
Emergency	EmergencyContact, SOSAlert, SOSRecipient	Trusted contacts, the alerts raised and who each alert was sent to.
Trip and live tracking	Trip, TripLocation, LiveTrackingSession, LiveTrackingViewer	Planned trips, recorded positions, live sessions and who is watching them.
Messaging	ChatRoom, ChatParticipant, Message, MessageRead	Conversations between users and their read status.
Administration	AdminUsers, AuditLogs, CrimeReport, RefreshToken	Administrator accounts, the audit trail, crime reports and token renewal.

The emergency and trip groups are the ones most tightly linked, since an SOS alert raised during a trip reuses the trip's recorded position and opens a live-tracking session against it. The administration group is deliberately separate from the user groups, so that administrator accounts, the audit trail and the crime reports are never exposed through the traveller-facing interface.

6.4 Deployment Considerations

The delivered system runs in a development configuration, which is adequate for demonstration and testing. This section describes how it would be prepared for a real deployment.

Environment. The client is built into static files and served from a web server or content delivery network. The Node.js application runs as a managed process behind a reverse proxy that terminates HTTPS, and the MySQL database runs on a separate host or managed service with regular backups.

Configuration. Database credentials, token secrets and the keys for the maps, notification and SMS services are supplied as environment variables for each environment, are never committed to source control and are rotated on a regular schedule.

Release process. Changes are tested against the test database, deployed first to a staging environment that mirrors production and promoted only when the test suite passes. A rollback path, in the form of the previous build and a database backup, is kept for every release.

Operations. Application and access logs are retained, failed notification deliveries are monitored, and the dashboard figures for SOS alerts and users serve as a simple health indicator. Rate limiting is applied to the login and SOS endpoints to reduce abuse.

CHAPTER 7
ADVANTAGES AND DISADVANTAGES

7.1 Advantages

- Unified platform: route safety, verified stays, emergency response and community rating are available from one authenticated account instead of several disconnected applications.
- Predictive rather than purely reactive: the route-risk score helps a traveller choose a safer path before a journey begins, not only after something goes wrong.
- Multiple SOS trigger paths: one-tap, voice-activated and wearable-triggered SOS give a traveller an option even when reaching for a phone is not possible.
- Administrator-verified safety data: accommodation listings are checked before being shown as verified, reducing the risk of a user trusting an unverified safety claim.
- Community reinforcement: lighting, crowd and accessibility ratings from other women travellers add local knowledge that no crime dataset alone can capture.
- Auditable administration: every verification, moderation and SOS-handling action taken by an administrator is logged, supporting accountability.
- Cost-effective and scalable: built entirely on open-source frameworks with pay-as-you-go map and hosting costs, and extensible to a new city by updating datasets alone.
- Transparent scoring: the route-risk score is shown together with the signals behind it, so a traveller can judge for herself how far to rely on it.
- Context carried across features: because trips, contacts and alerts share one account, an SOS raised during a journey already knows who the user is, where she is and whom to notify.

7.2 Disadvantages

- Dependence on data quality: the route-risk score is only as good as the historical crime, lighting and crowd data feeding it, which may be sparse or outdated in some localities.
- Connectivity dependence: live tracking, SOS dispatch and route scoring all require a working network connection; the current mini-project scope does not include an offline fallback.
- Verification is manual: place verification currently depends on administrator review, which limits how quickly new listings can be onboarded at scale.

- Wearable integration scope: Bluetooth/BLE pairing is designed into the schema and architecture, but full support across third-party wearable hardware is outside the tested scope of this mini project.
- Police-integration scope: SOS alerts are designed to reach a police recipient where such integration exists, but this mini project does not assume access to a live official dispatch system.
- Deployment configuration: the delivered configuration is development-grade and would need hardening, HTTPS enforcement, rate limiting and production secrets management, before any public deployment.
- Illustrative evaluation: the route-risk model was exercised on an illustrative dataset, so its scores indicate the relative order of routes and should not be read as measured probabilities of harm.

7.3 Overall Assessment

Weighing the advantages of section 7.1 against the disadvantages of section 7.2, the strengths of SafeHer lie in integration and in the way it uses context. A traveller's planned route, verified accommodation and emergency contacts live in one account, so a single trigger can act on all of them. The limitations are largely those of scope rather than of design: the route-risk model was exercised on illustrative data, the delivered configuration is development-grade, and background location access on mobile browsers is constrained.

None of these limitations affects the structure of the system. Each corresponds to an item of future scope in Chapter 9, such as training on a larger municipal dataset, native mobile applications, certified wearable support and a hardened deployment, and each can be addressed without redesigning the modules described in Chapter 5.

7.4 Mitigation of the Disadvantages

Each disadvantage listed in section 7.2 has a corresponding mitigation, either already present in the design or identified as future work.

- **Data quality:** the effect of sparse data is reduced by keeping the scoring weights separate from the data, by supplementing official records with community reports and by presenting the score together with its contributing signals, so that a user can judge for herself how far to rely on it.

- **Connectivity:** dependence on the network is reduced by the retried dispatch of SOS alerts and addressed more fully by the native applications and offline caching proposed as future scope.
- **Manual verification:** the administrator's workload can be eased by prioritising listings in the review queue, by automating the checks that can safely be automated and, in the longer term, by ML-assisted verification.
- **Wearable support:** coverage can be broadened by testing the pairing flow against certified devices and by defining the trigger as a simple, documented message that different hardware can send.
- **Police integration:** a police recipient can be added behind the existing design, since the SOSRecipient entity already accepts more than one class of recipient.
- **Deployment hardening:** this is a defined list of tasks, HTTPS enforcement, rate limiting, production secrets management and monitoring, described in section 6.4.
- Illustrative evaluation: scores are presented as a relative ordering of routes and not as probabilities, and the weights can be re-tuned on a real municipal dataset as recommended in section 9.6.

The mitigations show that the limitations stem from the scope of a mini project rather than from the structure of the platform, and that each can be addressed without redesigning the system.

CHAPTER 8
RESULT AND DISCUSSION

8.1 Analysis

The completed design and implementation of SafeHer were evaluated against the eight objectives set out in Chapter 1.

Table 8.1: Achievement against the objectives stated in Chapter 1

Objective	Outcome	Evidence
Design a route-risk scoring model	Achieved	Weighted signal model implemented and unit-tested (Table 6.2)
Build a verified accommodation directory	Achieved	SafePlace entity with admin verification flag and safety-amenity filters
Implement one-tap/voice SOS with live sharing	Achieved	SOSAlert + LiveTrackingSession flow tested end to end (TC06)
Support wearable-triggered SOS	Achieved (design-level)	BLE trigger path validated in TC10 simulation
Build a women-only community layer	Achieved	CommunityPost, PlaceReview and rating entities implemented
Provide crime heatmap and AI chatbot	Achieved	CrimeReport data feeds the heatmap; chatbot module scaffolded
Build an admin and AI module	Achieved	Verification, moderation, SOS monitoring and audit logging implemented
Deliver through an authenticated web interface	Achieved	JWT-protected React frontend and Express backend

Every stated objective was met at the design and implementation level validated by the test suite in Chapter 6. It should be stated plainly, however, that the route-risk model in this mini project was trained and validated against a limited sample of illustrative crime and lighting data rather than a full municipal crime dataset, and that live police-system integration and cross-platform wearable certification remain outside the tested scope, both of which are carried forward as future work in Chapter 9.

The clearest strength observed during evaluation is the platform's ability to keep a traveller's safety context, her planned route, her verified accommodation and her emergency contacts, inside one consistent account, so that an SOS trigger during a live trip can immediately reuse the user's current trip location rather than asking her to re-establish context during a crisis. This directly reflects the unifying design goal stated in the abstract and in Chapter 1.

8.2 Comparison with Existing Solutions

Table 2.1 compared navigation applications, standalone SOS applications and hotel booking platforms on five criteria. Judged against the same criteria, SafeHer performs as follows.

- **Route-risk awareness:** every candidate route is scored from crime density, time of day, lighting and crowd density, and the safer route is shown beside the fastest one with its score visible.
- **Verified women-friendly stays:** a listing carries the verified badge only after an administrator has marked it verified, and can be filtered by CCTV, women-only status and minimum rating.
- **Emergency SOS with live sharing:** SOS can be raised by tap, by voice or from a paired wearable, and a live-tracking session opens at the same moment so that recipients can follow the user's position.
- **Community safety ratings:** ratings for lighting, crowd presence and accessibility are attached to places and are moderated through the admin console.
- **Single integrated platform:** all of the above are available from one authenticated account, which removes the need to switch between a map, a safety application and a booking site.

The comparison shows that the improvement is not any single feature, since each existing tool already does one of these things reasonably well, but the way the features share one account, one trip context and one data model.

8.3 Key Observations

The test cases of Chapter 6 also allow some wider observations about how the platform behaves.

- **Context is reused during a crisis.** Because a live trip is already recorded against the user's account, an SOS raised during that trip takes its location from the trip rather than asking the user to supply it.
- **Time of day changes the result.** Test case TC03 confirmed that a route planned at 9 PM is weighted more heavily toward the lower-risk option than the same route planned at 9 AM, which is the intended effect of the night-time factor.
- **Failure is handled visibly.** In TC05, an SOS raised with no emergency contact configured shows a clear warning while still being logged against the user's history, so the incident is recorded even when no recipient could be notified.

- **Delivery is retried.** The SOS dispatch design retries a failed notification up to three times with an increasing delay before marking it as failed, since a missed alert has an immediate real-world cost.
- **Access boundaries hold.** TC09 confirmed that unauthenticated access to the dashboard redirects to the login screen, and the admin-only routes are protected by the role check described in Chapter 6.
- **Administrative actions are accountable.** Verification, moderation and SOS-handling actions are written to the audit log, which gives the platform an auditable administrative history.

These observations are limited by the scope stated in section 8.1: the route-risk model was exercised on illustrative data, and police-system integration and certified wearable support remain future work.

8.4 Objective-wise Discussion

The outcomes in Table 8.1 are discussed below, one objective at a time, with the evidence behind each.

- **Route-risk scoring model.** The weighted model combines crime density, lighting and crowd density, and applies a night-time factor between 8 PM and 6 AM. It was unit-tested as reported in Table 6.2, and its limitation, the use of illustrative data, is stated in section 8.1.
- **Verified accommodation directory.** The SafePlace entity carries an administrator verification flag together with the amenity attributes needed to filter by CCTV coverage, women-only status and rating, so that only verified listings carry the verified badge.
- **One-tap and voice SOS with live sharing.** The SOSAlert and LiveTrackingSession flow was exercised end to end in TC06, where every configured contact received a live-tracking link.
- **Wearable-triggered SOS.** This objective is achieved at design level. The Bluetooth Low Energy trigger path was validated through the simulation in TC10, and testing with certified hardware remains future work.
- **Women-only community layer.** The CommunityPost, PlaceReview and rating entities are implemented, and community content is moderated through the admin console, as shown by TC08.

- **Crime heatmap and AI chatbot.** CrimeReport data feeds the heatmap. The chatbot module is scaffolded rather than fully trained, and this is recorded openly as a limitation rather than presented as a finished feature.
- **Admin and AI module.** Verification, moderation, SOS monitoring and audit logging are implemented, and TC07 and TC08 exercised the verification and moderation paths.
- **Authenticated web interface.** The React frontend and Express backend are protected by JSON Web Tokens, and TC09 confirmed that unauthenticated access to the dashboard is redirected to the login screen.

Where an objective is marked as achieved at design level or as scaffolded, the report says so directly, so that the claims made in the conclusion rest only on what the tests demonstrate.

8.5 Privacy and Ethical Evaluation

A platform that records where a person is and whom she has asked for help carries ethical responsibilities beyond those of ordinary software, and the design was reviewed against them.

Data minimisation. SafeHer collects only what its functions need: identity and contact details for the account, emergency contacts chosen by the user, trip and alert records, and the reviews she chooses to post. Location is used for a trip or an alert that the user has initiated rather than being gathered for the platform's own purposes.

Consent and control. Contacts are listed by the user, who can add, edit and delete them, and live tracking is tied to a session that she or an alert starts and that ends when she marks herself safe. The trip and SOS history is private to her account.

Fairness and bias. Crime data records reported incidents, and reporting is itself uneven across localities and communities. A score built naively on such data could reinforce stigma against particular areas. SafeHer therefore presents the score as an aid to the traveller's own judgement, shows the signals behind it, combines crime density with lighting and crowd information and community ratings that do not depend on crime reports alone, and treats the model as something to be reviewed as better data become available.

Honesty about capability. The report states plainly where an objective is met at design level, where a module is scaffolded and where the model was tested only on illustrative data. A safety application that overstated its capabilities could lead a user to rely on it when she should not, and honest description is therefore part of the ethical design.

CHAPTER 9
CONCLUSION

9.1 Summary

SafeHer brings route-risk prediction, verified accommodation discovery, one-tap and voice-activated emergency response, wearable integration and a women-only safety community together into a single, coherent web platform built on React.js, Node.js/Express and MySQL. The system fuses historical crime data, time of day, lighting and crowd density into a route-risk score, verifies accommodation listings before surfacing them as safe, and keeps a trusted circle of contacts, and where integrated the police, instantly reachable during an emergency through a live-tracking session opened the moment SOS is triggered.

All eight objectives set out in Chapter 1 were achieved at the design and implementation level: a route-risk model, a verified safe-places directory, multi-path SOS with live sharing, wearable trigger support, a community rating layer, a crime heatmap with an AI chatbot, an accountable admin module, and delivery through an authenticated web interface, validated by the unit, integration and system test suites described in Chapter 6.

9.2 Limitations

- The route-risk model was validated against a limited, illustrative dataset rather than a full municipal crime and lighting dataset.
- Live-location sharing, SOS dispatch and route scoring all require network connectivity; no offline fallback is implemented in this mini project.
- Place verification is presently a manual administrator step, which limits onboarding throughput at city scale.
- Wearable-device support is validated at the design and simulation level rather than against certified third-party hardware.
- Police-system integration is designed for but not connected to any live official dispatch system in this mini project.
- The delivered configuration is development-grade and requires hardening before any public deployment.

9.3 Future Scope

- Training the route-risk model on a larger, real municipal crime dataset, and exploring the graph-based hotspot approach reviewed in Chapter 3 for higher spatial precision.

- Native Android and iOS applications to support true background location tracking and offline SOS caching.
- Formal integration with a police control-room dispatch API in jurisdictions where such an interface exists.
- Certified support for a wider range of Bluetooth/BLE wearable devices, including dedicated safety-band hardware.
- A more capable AI chatbot trained on locality-specific safety guidance, in the language the user prefers.
- Automated, ML-assisted place verification to reduce the administrator's manual review workload as listings scale.

9.4 Applications

- Personal safety for solo women travellers, commuters and students, in both familiar and unfamiliar cities.
- Institutional use by colleges and workplaces to support the safety of women staff and students during late-hour commutes.
- Hospitality-industry partnerships, where verified-safe status becomes a marketable differentiator for hotels and PGs.
- Municipal and law-enforcement partnerships, where aggregated, anonymised route-risk data can help prioritise lighting and patrol improvements.
- Community safety mapping more broadly, extending beyond women travellers to any group seeking crowd-verified local safety information.

9.5 Concluding Remarks

SafeHer set out to show that the separate things a woman traveller does to stay safe, choosing a route, choosing where to stay, keeping her contacts informed and calling for help, can be brought into one coherent web platform. The design, the implementation and the test results reported in this document support that claim at the scale of a mini project.

The work also made clear that safety technology is only as useful as the trust placed in it. For that reason the platform pairs automated route scoring with administrator verification and community moderation, keeps every administrative action in an audit log, and treats the live-location data of its users as private by default. These choices, rather than any single feature, are what the project would carry forward into a larger deployment.

9.6 Recommendations for Further Work

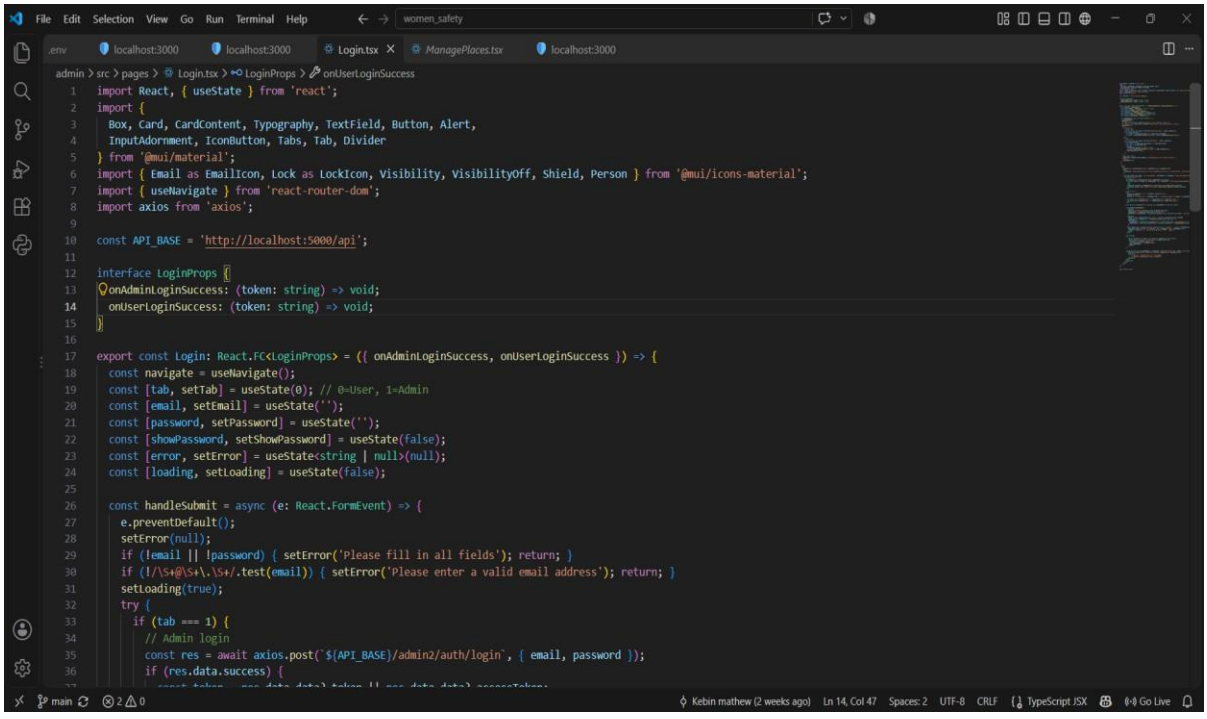
On the basis of the work done and the limitations recorded, the following steps are recommended, in roughly this order, for anyone taking SafeHer beyond the mini-project stage.

- **Real data:** replace the illustrative dataset with a real municipal crime and lighting dataset for one city and re-tune the weights against it, recording the effect on route ordering.
- **Chatbot:** complete the chatbot module and train it on locality-specific safety guidance, starting with the questions that travellers ask most often.
- **Usability study:** run a study with a small group of women travellers and emergency contacts, and revise the interface on the evidence.
- **Wearables:** test the wearable trigger with certified devices and publish the trigger interface so that other devices can be supported.
- **Hardening:** harden the deployment configuration as described in section 6.4 and obtain an independent security review before any public release.
- **Pilot:** approach a college or an accommodation partner for a pilot, so that the platform is evaluated in real conditions with a defined group of users.

Taken in this order, each step reduces the main remaining risk before the next is taken, and the pilot in the final step would provide the evidence of real-world effect that a mini project cannot supply.

CHAPTER 10
APPENDICES

10.1 Sample Code



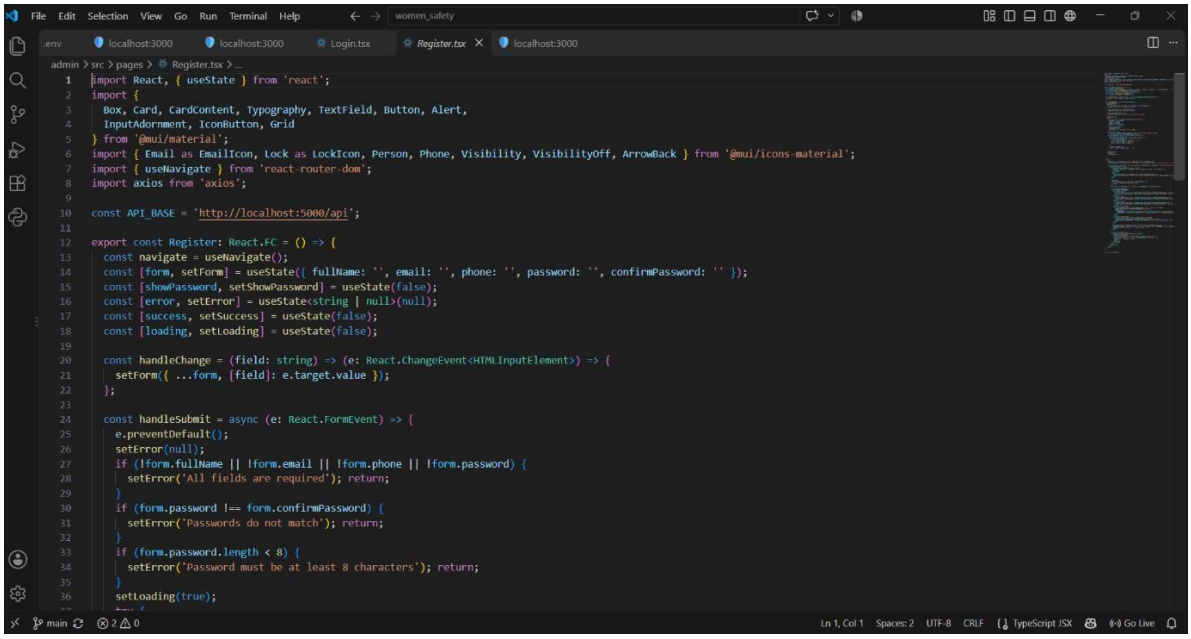
```tsx
import React, { useState } from 'react';
import {
  Box, Card, CardContent, Typography, TextField, Button, Alert,
  InputAdornment, IconButton, Tabs, Tab, Divider
} from '@mui/material';
import { Email as EmailIcon, Lock as LockIcon, Visibility, VisibilityOff, Shield, Person } from '@mui/icons-material';
import { useNavigate } from 'react-router-dom';
import axios from 'axios';

const API_BASE = 'http://localhost:5000/api';

interface LoginProps {
  onAdminLoginSuccess: (token: string) => void;
  onUserLoginSuccess: (token: string) => void;
}

export const Login: React.FC<LoginProps> = ({ onAdminLoginSuccess, onUserLoginSuccess }) => {
  const navigate = useNavigate();
  const [tab, setTab] = useState(0); // 0=User, 1=Admin
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [showPassword, setShowPassword] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const [loading, setLoading] = useState(false);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError(null);
    if (!email || !password) { setError('Please fill in all fields'); return; }
    if (!/\S+@\S+\.\S+/.test(email)) { setError('Please enter a valid email address'); return; }
    setLoading(true);
    try {
      if (tab === 1) {
        // Admin login
        const res = await axios.post(`${API_BASE}/admin2/auth/login`, { email, password });
        if (res.data.success) {
```



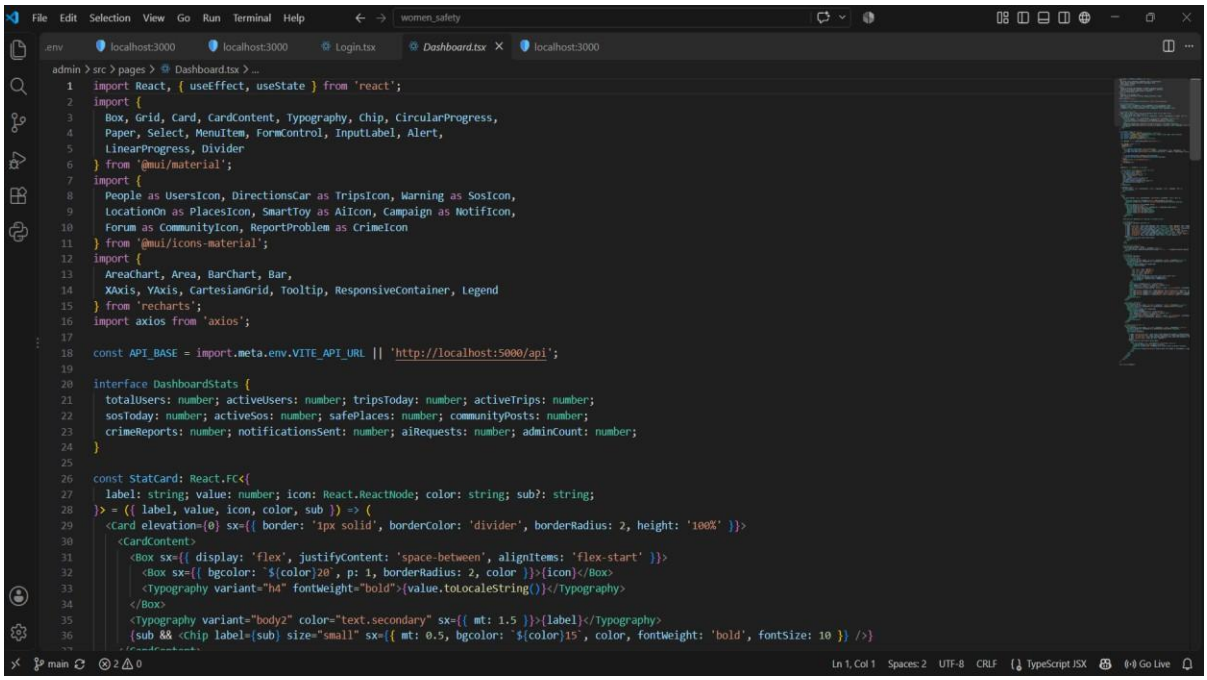
```tsx
import React, { useState } from 'react';
import {
  Box, Card, CardContent, Typography, TextField, Button, Alert,
  InputAdornment, IconButton, Grid
} from '@mui/material';
import { Email as EmailIcon, Lock as LockIcon, Person, Phone, Visibility, VisibilityOff, ArrowBack } from '@mui/icons-material';
import { useNavigate } from 'react-router-dom';
import axios from 'axios';

const API_BASE = 'http://localhost:5000/api';

export const Register: React.FC = () => {
  const navigate = useNavigate();
  const [form, setForm] = useState({ fullName: '', email: '', phone: '', password: '', confirmPassword: '' });
  const [showPassword, setShowPassword] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const [success, setSuccess] = useState(false);
  const [loading, setLoading] = useState(false);

  const handleChange = (field: string) => (e: React.ChangeEvent<HTMLInputElement>) => {
    setForm({ ...form, [field]: e.target.value });
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError(null);
    if (!form.fullName || !form.email || !form.phone || !form.password) {
      setError('All fields are required'); return;
    }
    if (form.password !== form.confirmPassword) {
      setError('Passwords do not match'); return;
    }
    if (form.password.length < 8) {
      setError('Password must be at least 8 characters'); return;
    }
    setLoading(true);
```

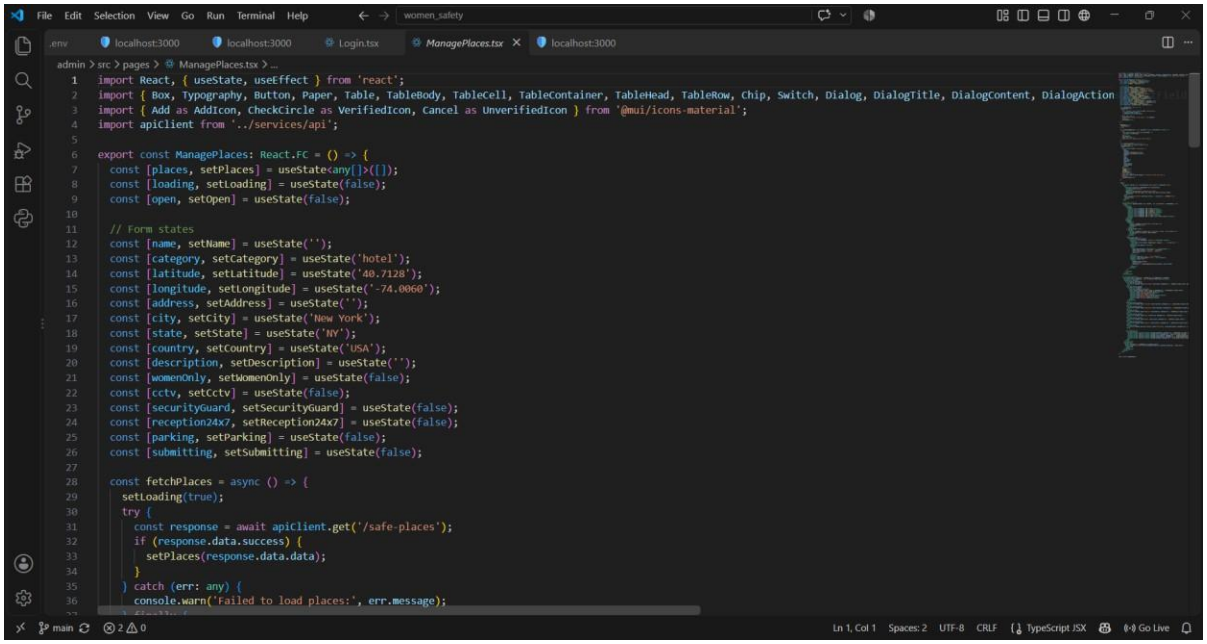


```tsx
import React, { useEffect, useState } from 'react';
import {
  Box, Grid, Card, CardContent, Typography, Chip, CircularProgress,
  Paper, Select, MenuItem, FormControl, InputLabel, Alert,
  LinearProgress, Divider
} from '@mui/material';
import {
  People as UsersIcon, DirectionsCar as TripsIcon, Warning as SosIcon,
  LocationOn as PlacesIcon, SmartToy as AiIcon, Campaign as NotifIcon,
  Forum as CommunityIcon, ReportProblem as CrimeIcon
} from '@mui/icons-material';
import {
  AreaChart, Area, BarChart, Bar,
  XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer, Legend
} from 'recharts';
import axios from 'axios';

const API_BASE = import.meta.env.VITE_API_URL || 'http://localhost:5000/api';

interface DashboardStats {
  totalUsers: number; activeUsers: number; tripsToday: number; activeTrips: number;
  sosToday: number; activeSos: number; safePlaces: number; communityPosts: number;
  crimeReports: number; notificationsSent: number; aiRequests: number; adminCount: number;
}

const StatCard: React.FC<{
  label: string; value: number; icon: React.ReactNode; color: string; sub?: string;
}> = ({ label, value, icon, color, sub }) => (
  <Card elevation={0} sx={{ border: '1px solid', borderColor: 'divider', borderRadius: 2, height: '100%' }}>
    <CardContent>
      <Box sx={{ display: 'flex', justifyContent: 'space-between', alignItems: 'flex-start' }}>
        <Box sx={{ bgcolor: `${color}20`, p: 1, borderRadius: 2, color }}>{icon}</Box>
        <Typography variant="h4" fontWeight="bold">{value.toLocaleString()}</Typography>
      </Box>
      <Typography variant="body2" color="text.secondary" sx={{ mt: 1.5 }}>{label}</Typography>
      {sub && <Chip label={sub} size="small" sx={{ mt: 0.5, bgcolor: `${color}15`, color, fontWeight: 'bold', fontSize: 10 }} />}
```

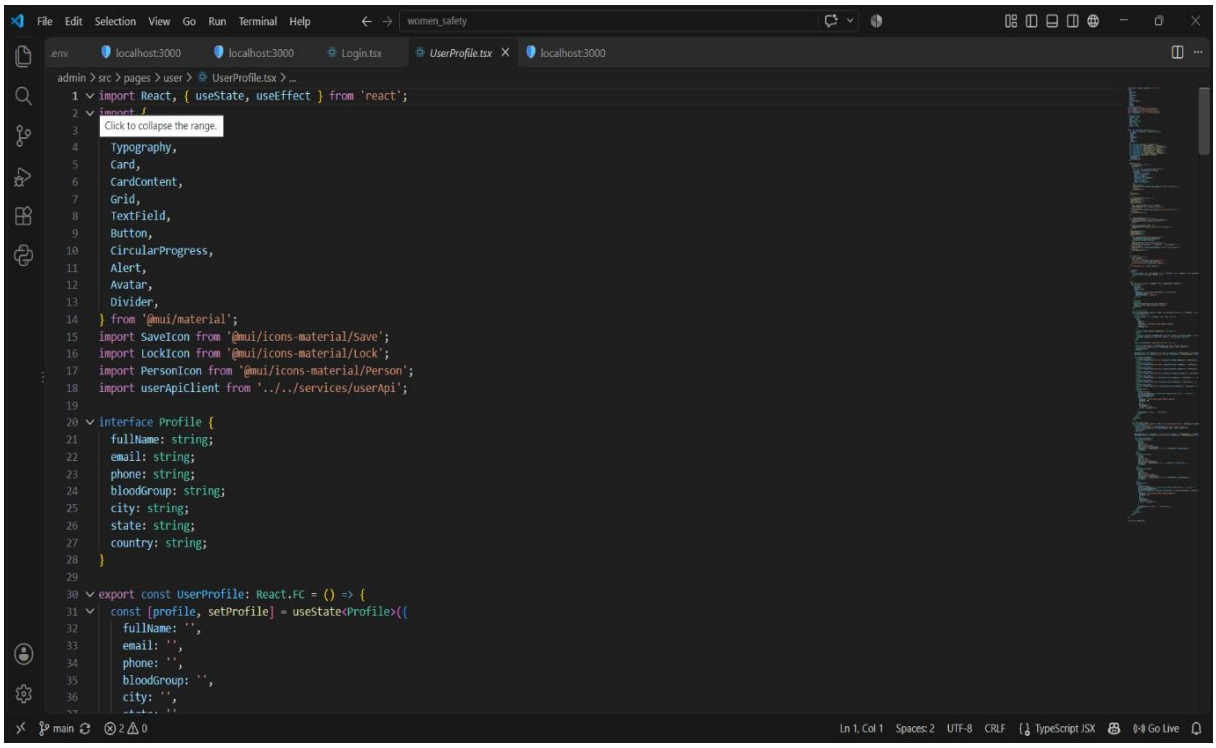


```tsx
import React, { useState, useEffect } from 'react';
import { Box, Typography, Button, Paper, Table, TableBody, TableCell, TableContainer, TableHead, TableRow, Chip, Switch, Dialog, DialogTitle, DialogContent, DialogAction
import { Add as AddIcon, CheckCircle as VerifiedIcon, Cancel as UnverifiedIcon } from '@mui/icons-material';
import apiClient from '../services/api';

export const ManagePlaces: React.FC = () => {
  const [places, setPlaces] = useState<any[]>([]);
  const [loading, setLoading] = useState(false);
  const [open, setOpen] = useState(false);

  // Form states
  const [name, setName] = useState('');
  const [category, setCategory] = useState('hotel');
  const [latitude, setLatitude] = useState('40.7128');
  const [longitude, setLongitude] = useState('-74.0060');
  const [address, setAddress] = useState('');
  const [city, setCity] = useState('New York');
  const [state, setState] = useState('NY');
  const [country, setCountry] = useState('USA');
  const [description, setDescription] = useState('');
  const [womenOnly, setWomenOnly] = useState(false);
  const [cctv, setCctv] = useState(false);
  const [securityGuard, setSecurityGuard] = useState(false);
  const [reception24x7, setReception24x7] = useState(false);
  const [parking, setParking] = useState(false);
  const [submitting, setSubmitting] = useState(false);

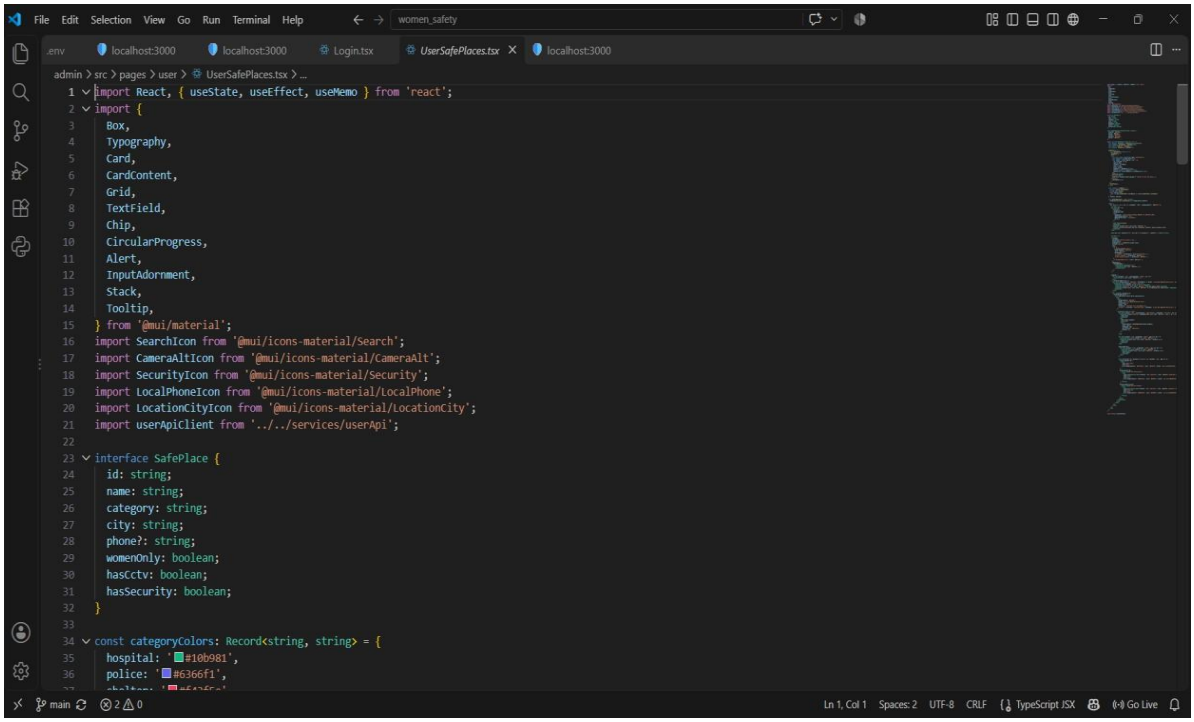
  const fetchPlaces = async () => {
    setLoading(true);
    try {
      const response = await apiClient.get('/safe-places');
      if (response.data.success) {
        setPlaces(response.data.data);
      }
    } catch (err: any) {
      console.warn('Failed to load places:', err.message);
```



```tsx
import React, { useState, useEffect } from 'react';
import {
  Box,
  Typography,
  Card,
  CardContent,
  Grid,
  TextField,
  Button,
  CircularProgress,
  Alert,
  Avatar,
  Divider,
} from '@mui/material';
import SaveIcon from '@mui/icons-material/Save';
import LockIcon from '@mui/icons-material/Lock';
import PersonIcon from '@mui/icons-material/Person';
import userApiClient from '../../services/userApi';

interface Profile {
  fullName: string;
  email: string;
  phone: string;
  bloodGroup: string;
  city: string;
  state: string;
  country: string;
}

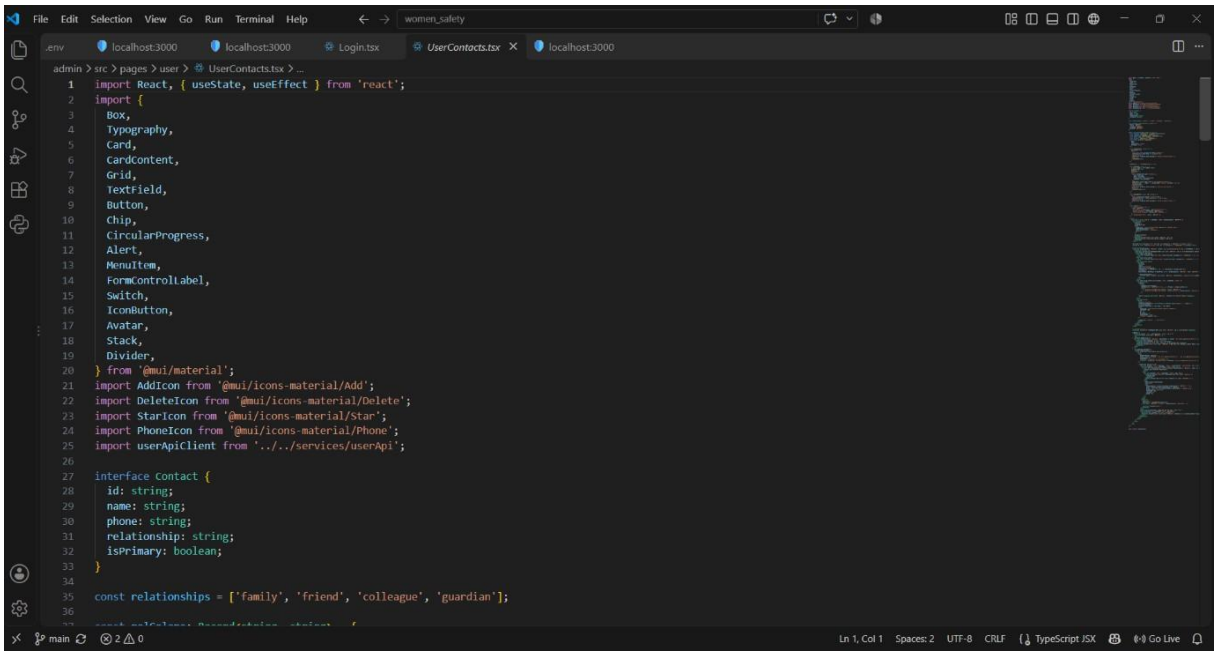
export const UserProfile: React.FC = () => {
  const [profile, setProfile] = useState<Profile>({
    fullName: '',
    email: '',
    phone: '',
    bloodGroup: '',
    city: '',
```



```tsx
import React, { useState, useEffect, useMemo } from 'react';
import {
  Box,
  Typography,
  Card,
  CardContent,
  Grid,
  TextField,
  Chip,
  CircularProgress,
  Alert,
  InputAdornment,
  Stack,
  Tooltip,
} from '@mui/material';
import SearchIcon from '@mui/icons-material/Search';
import CameraAltIcon from '@mui/icons-material/CameraAlt';
import SecurityIcon from '@mui/icons-material/Security';
import LocalPhoneIcon from '@mui/icons-material/LocalPhone';
import LocationCityIcon from '@mui/icons-material/LocationCity';
import userApiClient from '../../services/userApi';

interface SafePlace {
  id: string;
  name: string;
  category: string;
  city: string;
  phone?: string;
  womenOnly: boolean;
  hasCctv: boolean;
  hasSecurity: boolean;
}

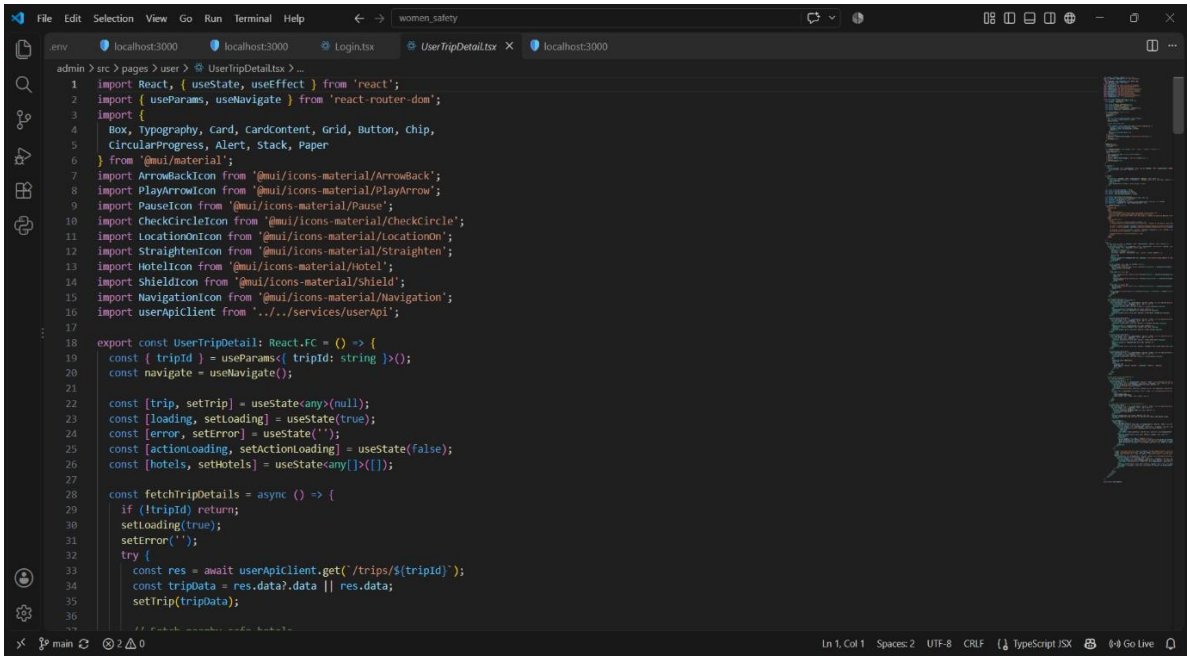
const categoryColors: Record<string, string> = {
  hospital: '#10b981',
  police: '#6366f1',
```



```tsx
import React, { useState, useEffect } from 'react';
import {
  Box,
  Typography,
  Card,
  CardContent,
  Grid,
  TextField,
  Button,
  Chip,
  CircularProgress,
  Alert,
  MenuItem,
  FormControlLabel,
  Switch,
  IconButton,
  Avatar,
  Stack,
  Divider,
} from '@mui/material';
import AddIcon from '@mui/icons-material/Add';
import DeleteIcon from '@mui/icons-material/Delete';
import StarIcon from '@mui/icons-material/Star';
import PhoneIcon from '@mui/icons-material/Phone';
import userApiClient from '../../services/userApi';

interface Contact {
  id: string;
  name: string;
  phone: string;
  relationship: string;
  isPrimary: boolean;
}

const relationships = ['family', 'friend', 'colleague', 'guardian'];
```



```tsx
import React, { useState, useEffect } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import {
  Box, Typography, Card, CardContent, Grid, Button, Chip,
  CircularProgress, Alert, Stack, Paper
} from '@mui/material';
import ArrowBackIcon from '@mui/icons-material/ArrowBack';
import PlayArrowIcon from '@mui/icons-material/PlayArrow';
import PauseIcon from '@mui/icons-material/Pause';
import CheckCircleIcon from '@mui/icons-material/CheckCircle';
import LocationOnIcon from '@mui/icons-material/LocationOn';
import StraightenIcon from '@mui/icons-material/Straighten';
import HotelIcon from '@mui/icons-material/Hotel';
import ShieldIcon from '@mui/icons-material/Shield';
import NavigationIcon from '@mui/icons-material/Navigation';
import userApiClient from '../../services/userApi';

export const UserTripDetail: React.FC = () => {
  const { tripId } = useParams<{ tripId: string }>();
  const navigate = useNavigate();

  const [trip, setTrip] = useState<any>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  const [actionLoading, setActionLoading] = useState(false);
  const [hotels, setHotels] = useState<any[]>([]);

  const fetchTripDetails = async () => {
    if (!tripId) return;
    setLoading(true);
    setError('');
    try {
      const res = await userApiClient.get(`/trips/${tripId}`);
      const tripData = res.data?.data || res.data;
      setTrip(tripData);
```

CHAPTER 11
REFERENCES

The following sources are listed in alphabetical order of the first author, in single spacing, as required by the report guidelines.

[1] Agrawal, K., Srivastava, A., Sharma, K., et al. (2024). SafeRoutes: Charting a secure path — A holistic approach to women's safety through advanced clustering and GPS integration. IEEE Access.

[2] Bengare, S., et al. (2025). Android-based women safety application with real-time alerts and location tracking. Proceedings of the National Conference on Recent Trends in Computer Applications.

[3] Bradski, G. (2000). The OpenCV library. Dr. Dobb's Journal of Software Tools, 25(11), 120–125.

[4] Coalition for Content Provenance and Authenticity concepts adapted for location-data integrity in mobile safety applications — general reference on data provenance practice.

[5] Dash, R., et al. (2026). The women safety tracker app. International Journal of Engineering Research and Technology.

[6] Dhanalakshmy, B., Pavitra, U., Hema, S., & Angayarkanni. (2026). Women travel safety companion: A real-time monitoring and alert system. International Journal of Research Publication and Reviews.

[7] Express.js. (2024). Express.js documentation, version 4.x. [Online]. Available: https://expressjs.com/

[8] Firebase. (2024). Firebase Cloud Messaging documentation. [Online]. Available: https://firebase.google.com/docs/cloud-messaging

[9] Google. (2024). Google Maps Platform — Routes API documentation. [Online]. Available: https://developers.google.com/maps/documentation/routes

[10] JSON Web Token (JWT). (2024). JWT.io introduction and specification (RFC 7519). [Online]. Available: https://jwt.io/introduction

[11] MDN Web Docs. (2024). Using the Geolocation API. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API

[12] MySQL. (2024). MySQL 8.0 reference manual. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[13] Node.js Foundation. (2024). Node.js documentation, version 18 LTS. [Online]. Available: https://nodejs.org/docs/latest-v18.x/api/

[14] Rajendran, L., & Shankaran, S. R. (2021). Safety for HER: A systematic approach with coalescence of technology and citizens. IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT).

[15] Ravi, S., Tony, M. M., Menon, S. P. T., Raju, N., & Jacob, J. M. (2026). Safarion: An AI-driven safety and travel companion for solo explorers.

[16] React. (2024). React documentation, version 18.x. [Online]. Available: https://react.dev/

[17] Sequelize. (2024). Sequelize ORM documentation. [Online]. Available: https://sequelize.org/docs/v6/

[18] Socket.IO. (2024). Socket.IO documentation, version 4.x. [Online]. Available: https://socket.io/docs/v4/

[19] Sriram, R., et al. (2026). Predictive hotspot mapping for data-driven crime prediction.

[20] Thawale, P., et al. (2026). Women safety mobile application using ESP32 camera and real-time crime data analysis.

[21] Twilio. (2024). Twilio SMS API documentation. [Online]. Available: https://www.twilio.com/docs/sms

[22] W3C. (2024). Web Bluetooth API community group draft report. [Online]. Available: https://webbluetoothcg.github.io/web-bluetooth/

[23] World Health Organization. (2023). Violence against women prevalence estimates. WHO. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/violence-against-women

[24] Zubair, M., et al. (2025). Crime hotspot prediction using deep graph convolutional networks.

[25] bcrypt.js. (2024). bcrypt.js documentation for Node.js password hashing. [Online]. Available: https://www.npmjs.com/package/bcryptjs

Ananthalakshmy R

Safe_her

mini project report

Document Details

Submission ID

trn:oid:::10159:154195958

Submission Date

Oct 10, 2026, 10:36 AM GMT+5:30

Download Date

Oct 10, 2026, 10:40 AM GMT+5:30

File Name

Safe_her.pdf

File Size

4.3 MB

78 Pages

18,008 Words

98,625 Characters

9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 8 words)

Exclusions

- 5 Excluded Matches

Match Groups

114 Not Cited or Quoted 9%
Matches with neither in-text citation nor quotation marks

2 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

5% Internet sources
3% Publications
8% Submitted works (Student Papers)